جامعة ٦ أكتوبر
October 6 University

علوم الحاسب ونظم المعلومات
Info Sys & Comp Science

Graduation Project

RAGShield - A Zero-Trust Multi-Layer Security Framework for Secure Retrieval-Augmented Generation Systems in Healthcare

Team Members :

Name	ID	Department
Mohab Nasser Abdelkader	23013228	NT
Mahmoud Abdelrazik Abdelwahab	23012776	NT
Mohamed Ahmed Mohamed	23017501	AI
Manar Abdelbaky Hassan	23012850	AI
Nada Mohamed Zainelaabedin	23012744	CS
Abdelrahman Mohamed Nour	23013290	CS
Omar Mohamed Elsayed	23016131	CS

Supervised By:

Dr. Reham Hussien

Co - Supervisors:

Academic Year 2026–2027

Table of Contents

List of Figures

List of Tables

CHAPTER ONE

Introduction

1.1 Introduction

Large Language Models (LLMs) have become increasingly integrated into enterprise applications, providing advanced capabilities for natural language understanding and generation. However, conventional LLMs have limitations when dealing with private, domain-specific, or continuously changing information. Retrieval-Augmented Generation (RAG) addresses these limitations by combining language generation with information retrieval, allowing models to retrieve relevant information from external knowledge bases and use it to generate more relevant and grounded responses.

Despite these advantages, the integration of external knowledge sources and retrieval mechanisms introduces new security and privacy risks. RAG systems may be exposed to threats such as prompt injection, malicious document ingestion, corpus or retrieval poisoning, unauthorized access to sensitive information, privacy leakage, and hallucinated or untrusted responses. These risks can affect different stages of the RAG pipeline, from user input and document ingestion to retrieval, generation, and output processing.

These challenges become particularly critical in healthcare environments, where RAG systems may interact with sensitive patient information and electronic medical records. Unauthorized access, disclosure of personally identifiable information (PII), and leakage of protected health information (PHI) can have serious consequences and require strong security and privacy controls.

This chapter therefore reviews the existing literature on RAG systems and their security challenges. The technical foundations of Large Language Models and Retrieval-Augmented Generation, together with the RAG architecture and pipeline, were established in Chapter One and are not repeated here. The chapter begins instead with the methodology used to identify and select the reviewed studies, including the research questions, the databases searched, the search strategy, the inclusion and exclusion criteria, and the outcome of the selection process. It then summarises the threat landscape reported in the selected literature before analysing the selected studies themselves, organised by security theme. Each theme concludes with a discussion that draws together what the studies collectively demonstrate and what they leave unresolved. The chapter closes by consolidating these observations into a set of research gaps and by positioning the proposed RAGShield framework against them.

1.2 Technical Background

This section introduces the technical concepts required to understand the remainder of this report. It describes what Large Language Models are and why their reliance on static training data motivates retrieval augmentation, defines Retrieval-Augmented Generation, and presents the RAG architecture as a multi-stage pipeline. The pipeline described here is the structure that the security analysis in Chapter Two and the framework designed in Chapter Three are organised around, since each of its stages constitutes a distinct trust boundary.

1.2.1 Large Language Models (LLMs)

Definition and Overview

Large Language Models (LLMs) are deep learning models trained on large-scale text datasets to understand and generate human language. Most modern LLMs are based on the Transformer architecture [9], which uses attention mechanisms to capture relationships between tokens and understand context.

Core Capabilities

LLMs can perform various language-related tasks, including:

- Text generation and understanding
- Question answering and summarization
- Translation and code generation
- Instruction following and reasoning

Limitations

Despite their capabilities, LLMs have several limitations:

- **Knowledge Cutoff:** Their knowledge is mainly limited to their training data and may not include recent information.
- **Hallucination:** They may generate plausible but incorrect or unsupported information.
- **Domain-Specific Gaps:** They may lack specialized or private knowledge.
- **Limited External Access:** They cannot directly access private databases or real-time information without additional mechanisms.
- **Bias and Limited Transparency:** Their outputs may reflect training-data biases, while the sources behind specific responses are difficult to determine.

These limitations motivate the use of RAG, which connects LLMs to external knowledge sources to provide relevant and up-to-date information during generation. However, this integration also introduces additional security and privacy challenges.

1.2.2 Retrieval-Augmented Generation (RAG)

Definition

RAG is an architecture that combines information retrieval with LLMs [1]. Instead of relying only on the knowledge learned during training, RAG retrieves relevant information from an external knowledge base and provides it to the LLM as context. This enables the system to use updated, private, and domain-specific information without retraining the model.

Core Concept and Process

RAG separates knowledge retrieval from language generation. When a user submits a query, the system retrieves relevant documents or document chunks from an external knowledge base and combines them with the query to construct the context provided to the LLM.

The process can be summarized as:

User Query → Retrieval → Relevant Documents → Context Construction → LLM → Response

This approach helps ground generated responses in external information and allows organizations to control the knowledge available to the system.

Key Advantages

- **External Knowledge:** Supports private, domain-specific, and updated information.
- **Improved Grounding:** Provides relevant context to support generated responses.
- **Knowledge Updating:** Documents can be updated without retraining the LLM.
- **Source Verification:** Retrieved documents can provide evidence for generated responses.
- **Knowledge Control:** Organizations can control the sources available to the system.

However, introducing external data and retrieval components also creates additional security risks. Malicious content, retrieval manipulation, unauthorized access, and untrusted information can affect the context provided to the LLM. Therefore, securing the complete RAG pipeline is essential.

1.2.3 RAG Architecture and Pipeline

A typical RAG system consists of two main stages: knowledge indexing and query processing. The first prepares external documents for retrieval, while the second retrieves relevant information and provides it to the LLM for response generation.

1. Document Ingestion and Preprocessing

Documents are collected from external sources and processed to extract their text and relevant metadata.

2. Text Chunking

Large documents are divided into smaller chunks to facilitate efficient embedding and retrieval while preserving relevant context.

3. Embedding Generation

Each chunk is converted into a numerical vector using an embedding model. These vectors represent the semantic meaning of the text and enable similarity-based retrieval.

4. Vector Database

The generated embeddings and their metadata are stored in a vector database or similarity-search index, enabling efficient retrieval of semantically relevant content.

5. Query Processing and Retrieval

The user query is converted into an embedding and compared with stored document embeddings. The most relevant chunks are retrieved based on their similarity to the query. Some systems may also use re-ranking to improve retrieval relevance.

6. Context Construction

The retrieved chunks are combined with the user query and required system instructions to construct the context provided to the LLM.

7. LLM Generation

The LLM processes the query and retrieved context to generate the final response.

8. Response Delivery

The generated response is returned to the user, optionally with references to the retrieved sources for verification.

The overall pipeline can be summarized as:

Documents → Preprocessing → Chunking → Embeddings → Vector Store

User Query → Query Embedding → Retrieval → Context Construction → LLM → Response

The modular nature of RAG allows its components to be independently modified or replaced. However, each component can also introduce security and privacy risks, making the architecture important for understanding the threats discussed in the following sections.

1.3 Security Challenges in Healthcare RAG Systems

Although LLMs have their own security challenges, integrating external knowledge sources introduces additional risks across the RAG pipeline. Security must therefore address not only the LLM but also document ingestion, storage, retrieval, context construction, and response generation. These risks are amplified in healthcare deployments, where the knowledge base may contain patient records and clinical documentation.

A key challenge is the presence of multiple trust boundaries. User queries, external documents, retrieved content, and generated responses may have different levels of trust. A vulnerability at any stage can affect the confidentiality, integrity, or reliability of the system.

RAG also has a broader and more dynamic attack surface than standalone LLMs. Malicious or manipulated content may enter the knowledge base and influence future responses when retrieved. Similarly, retrieval manipulation can cause the LLM to receive misleading or unauthorized information without modifying the model itself.

Another important challenge is data sensitivity, particularly when RAG systems process private or healthcare-related information. Strong access control and privacy protection are required to ensure that users can retrieve only the information they are authorized to access.

Therefore, RAG security requires a multi-layer approach covering input validation, document integrity, retrieval security, access control, privacy, and output validation. The major threats resulting from these challenges, including prompt injection, corpus poisoning, privacy leakage, unauthorized access, retrieval manipulation, and hallucination, are discussed in the following section.

1.4 Problem Statement

Current RAG security research addresses individual threats, such as prompt injection, corpus poisoning, privacy leakage, unauthorized retrieval, or output hallucination, largely in isolation. Consequently, a RAG system that is defended against one class of attack may remain fully exposed to another. This fragmentation creates a critical problem for sensitive application domains: there is currently no unified, practical security framework capable of protecting a RAG pipeline across all of its stages, that is, input handling, document ingestion, retrieval, access control, generation, and output delivery, while remaining efficient enough for real-world deployment. This problem is particularly acute in healthcare applications of RAG, where systems must simultaneously guarantee data confidentiality, enforce role- and attribute-based access to medical records, maintain the integrity of the underlying knowledge base, and ensure that generated responses do not disclose sensitive information or reflect manipulated retrieved content. Addressing these requirements with a set of disconnected, single-purpose tools is insufficient; a coordinated, zero-trust, multi-layer security architecture is required.

1.5 Research Aim and Objectives

Research Aim. The aim of this project is to design and evaluate RAGShield, a zero-trust, multi-layer security framework that provides coordinated protection across the complete Retrieval-Augmented Generation lifecycle, with healthcare as the primary application context.

Research Objectives. In support of this aim, the research pursues the following five objectives:

- **RO1.** To design an end-to-end zero-trust security architecture for healthcare RAG systems.
- **RO2.** To integrate coordinated security controls across the input, retrieval, privacy and access-control, and output stages of the RAG lifecycle.
- **RO3.** To implement monitoring and auditing mechanisms across the RAG lifecycle in order to support security visibility and accountability.
- **RO4.** To evaluate the proposed framework against relevant security threats and unauthorized-access scenarios.
- **RO5.** To assess the effect of the proposed security mechanisms on retrieval quality and overall system performance.

These objectives are stated at the level of the research contribution rather than at the level of individual mechanisms. The specific technical controls through which they are realised,

including input classification, integrity verification, sensitive-data masking, trust-aware retrieval, role- and attribute-based authorisation, and output validation, are design decisions and are presented in Chapter Three rather than treated as objectives in their own right.

1.6 Research Scope

This project focuses on the design, implementation, and evaluation of a multi-layer security framework for RAG systems, using a healthcare-oriented use case as the primary application context. The scope covers the security-relevant stages of the RAG lifecycle, namely input processing, document ingestion, embedding and vector storage, retrieval, context construction, and response generation and validation. The project does not aim to design a new foundation LLM or a new retrieval algorithm from first principles; rather, it aims to wrap and coordinate security controls around an existing RAG pipeline. Similarly, while the framework is evaluated using a healthcare-oriented dataset and threat scenarios, its underlying architecture is intended to generalize to other sensitive domains, such as finance and enterprise information systems, that share comparable confidentiality and integrity requirements.

1.7 Significance of the Study

This project contributes to the growing body of research on trustworthy and secure AI systems by addressing a gap that is not resolved by any single existing approach: the lack of an integrated, end to-end security architecture for RAG. By combining input security, document integrity, privacy protection, retrieval trust evaluation, dynamic access control, output validation, and continuous monitoring within a single coordinated framework, RAGShield offers a more complete and practical security posture than approaches that secure only one stage of the pipeline. In sensitive domains such as healthcare, where RAG systems increasingly support clinical decision-making, the significance of this work lies in enabling organizations to adopt RAG-based AI assistants with greater confidence that patient data remains protected, retrieved information remains trustworthy, and generated responses remain reliable, without incurring the prohibitive computational overhead associated with purely cryptographic solutions.

1.8 Report Organization

This chapter has established the technical background required for the remainder of the report. Chapter Two presents a structured review of existing research on RAG security, privacy, and

reliability. It documents the review methodology, analyses the selected studies by security theme, and derives the research gaps that motivate this project. Chapter Three presents the overall RAGShield architecture, the system requirements, and the detailed design models of the proposed framework. Subsequent chapters describe the implementation of the framework, the experimental setup and evaluation methodology, the results obtained, and the conclusions and directions for future work.

1.9 Chapter Summary

This chapter introduced the motivation behind securing Retrieval-Augmented Generation systems, highlighting how the retrieval of external knowledge, while improving factuality and domain relevance, also introduces new security and privacy risks that are especially critical in sensitive domains such as healthcare. It then presented the technical background required to follow the remainder of the report, covering Large Language Models and their limitations, the definition and operation of Retrieval-Augmented Generation, and the RAG architecture as a multi-stage pipeline whose individual components each constitute a security-relevant trust boundary. The security challenges specific to healthcare RAG deployments were then examined. The problem statement identified the lack of a unified, end-to-end security framework for RAG as the central challenge addressed by this project. The chapter then stated the research aim and five research objectives, together with the scope and significance of the proposed RAGShield framework, and described how the remainder of the report is organized. The following chapter reviews the related literature in detail and establishes the research gaps that shape the design of RAGShield.

CHAPTER TWO

Literature Review

2.1 Literature Review Methodology

This literature review follows a structured, systematic approach to identify and analyze existing research related to the security of RAG systems. The methodology is designed to ensure comprehensive coverage of relevant literature while maintaining rigorous selection criteria. The review focuses on security and privacy challenges across all stages of the RAG pipeline, including user input validation, document ingestion and storage, information retrieval, access control enforcement, context construction, response generation, and output verification.

The literature review process consists of six key components: defining research questions that guide the investigation, developing a comprehensive search strategy using controlled keywords and Boolean operators, selecting appropriate academic databases known for peer-reviewed publications in computer science and security, establishing clear inclusion and exclusion criteria to ensure relevance and quality, systematically screening identified studies through title/abstract review and full-text analysis, and documenting the selection process for reproducibility and transparency. These steps provide a systematic and transparent basis for analyzing previous research and identifying limitations in existing security mechanisms for RAG systems.

2.1.1 Research Questions

The literature review was guided by a comprehensive set of research questions designed to systematically investigate the security aspects of RAG systems, existing threats, proposed mitigation techniques, and limitations of current approaches. These research questions provide structured direction for analyzing the existing literature and identifying areas where further research and innovation are required. The main research questions are:

RQ1: What are the fundamental concepts, core components, and architectural designs of Retrieval-Augmented Generation systems?

RQ2: What are the major security threats and vulnerabilities affecting RAG-based systems across different pipeline stages (input, ingestion, retrieval, generation, output)?

RQ3: How can attacks such as prompt injection, corpus/retrieval poisoning, retrieval manipulation, and unauthorized information access affect RAG systems, and what are their real-world consequences, particularly in healthcare environments?

RQ4: What detection, prevention, and mitigation approaches have been proposed in existing literature to address security threats in RAG systems, and what techniques do they employ?

RQ5: What challenges and limitations remain unresolved or inadequately addressed in existing RAG security solutions, and what protection gaps exist?

RQ6: How can a multi-layer, end-to-end security framework improve the protection, reliability, trustworthiness, and regulatory compliance of RAG systems, particularly when handling sensitive domains such as healthcare with strict HIPAA and GDPR requirements?

RQ7: What are the performance, scalability, and usability trade-offs when implementing comprehensive security mechanisms in RAG systems, and how can these be balanced with practical deployment requirements?

These research questions provide a structured basis for analyzing the existing literature, identifying critical gaps, and establishing the need for comprehensive RAG security solutions that address multiple threat categories simultaneously.

2.1.2 Academic Databases and Sources Searched

The searches described in the following section were conducted across academic databases and digital libraries that index peer-reviewed research in artificial intelligence, natural language processing, information retrieval, and cybersecurity, together with recognised preprint repositories. Because RAG security is a rapidly developing field in which a substantial proportion of the relevant work appears first as a preprint or in conference proceedings, both peer-reviewed venues and technically substantial preprints were searched, consistent with the publication criteria stated in Section 2.2.4.

Table 2.1 lists the sources from which the finally selected studies were retrieved. These are the sources that can be confirmed from the publication venues of the selected studies themselves.

Table 2.1: Sources from Which the Selected Studies Were Retrieved

Source	Type	Role in the Review	Selected studies retrieved from this source
arXiv	Preprint repository	Preprint retrieval	TrustRAG; CRAG; PoisonedRAG
USENIX Security Symposium proceedings	Peer-reviewed conference	Published paper retrieval	PoisonedRAG
ACL Anthology	Peer-reviewed conference	Conference/workshop paper retrieval	Zeng et al.; MiRAG
OpenReview	Peer-reviewed workshop proceedings	Workshop paper retrieval	SecureRAG
ResearchGate	Research-sharing platform	Access to paper	ArabGuard

2.1.3 Search Strategy

The search strategy focused on research addressing RAG architectures and the security challenges associated with retrieval-augmented systems. Search terms were constructed by combining keywords related to RAG with terms describing security, privacy, retrieval, and specific attack classes.

Representative search queries included combinations such as:

- "Retrieval-Augmented Generation" AND "security"
- "RAG" AND "prompt injection"
- "RAG" AND "data poisoning"
- "RAG" AND "privacy"
- "RAG" AND "information leakage"
- "RAG" AND "access control"
- "RAG" AND "retrieval security"
- "RAG" AND "trust"
- "RAG" AND "retrieval poisoning"

Additional searches were performed using terms identified during the initial review of relevant literature. This iterative process helped identify studies addressing both direct security mechanisms and related retrieval or reliability techniques.

2.1.4 Inclusion Criteria

Studies were included in the literature review if they satisfied the following criteria:

Content Criteria:

- Directly address RAG ,LLMs, information retrieval, or closely related architectures.
- Investigate security threats, vulnerabilities, attacks, or risks relevant to RAG or LLM-based systems.
- Propose or evaluate security mechanisms, detection techniques, defense methods, or mitigation strategies.
- Address privacy, data protection, access control, or information leakage in AI/ML systems.
- Investigate prompt injection, adversarial attacks, corpus poisoning, document manipulation, or retrieval-related attacks.
- Examine privacy risks such as PII/PHI leakage or other forms of sensitive information disclosure.

- Investigate hallucination, factual consistency, grounding, or retrieval quality when relevant to RAG security and reliability.
- Present foundational concepts or techniques that are necessary for understanding LLMs, RAG, embeddings, retrieval, or related technologies.

Publication Criteria:

- Peer-reviewed conference papers and journal articles from reputable venues relevant to artificial intelligence, natural language processing, information retrieval, privacy, and cybersecurity.
- Relevant and technically substantial preprints from recognized research repositories such as arXiv, particularly for recent developments in RAG and LLM security.
- Technical reports, standards, or security guidelines from recognized organizations such as NIST and OWASP when they provide relevant security guidance or technical contributions.
- Academic theses or dissertations when they provide substantial and directly relevant research contributions.

Timeline Criteria:

- Publications from 2023 onwards were prioritized to cover the development of modern LLM and RAG technologies.
- Recent publications were prioritized to reflect the rapidly evolving nature of RAG and LLM security research.
- Earlier foundational studies were included when they provided essential concepts or techniques directly relevant to the research topic.

Quality Criteria:

- Studies must provide a clear research objective and sufficient methodological or technical detail.
- Studies should provide empirical evaluation, theoretical analysis, or other evidence supporting their conclusions.
- Studies with publicly available code, datasets, or detailed implementation information were preferred when applicable, but the absence of such resources was not considered an automatic exclusion criterion.
- Review and survey papers were included when they provided comprehensive coverage or established important foundations for the research topic.

2.1.5 Exclusion Criteria

Studies were excluded from the literature review if they met any of the following criteria:

Topic Exclusions:

- Focus exclusively on LLM performance, training, or efficiency without security/privacy component
- Purely theoretical cryptography or security papers without AI application

Publication Quality Exclusions:

- Non-peer-reviewed blog posts, forums, or informal publications
- Papers with fundamental methodological flaws or insufficient evaluation
- Duplicate or near-duplicate publications (kept most recent/complete version)

Timeline Exclusions:

- Publications before 2023 (except foundational papers explicitly cited as essential)

2.1.6 Study Selection Process

The study selection process followed a structured multi-phase approach to systematically identify and evaluate publications relevant to the research topic.

Phase 1: Search and Initial Identification

- Relevant publications were identified using the predefined search strategy.
- Retrieved publications were collected along with their bibliographic information, including title, authors, publication venue, and year.
- Duplicate publications were identified and removed before the screening process.

Phase 2: Title and Abstract Screening

- The titles and abstracts of the retrieved publications were screened according to the predefined inclusion and exclusion criteria.
- Publications clearly unrelated to RAG systems, LLM security, privacy, or the research objectives were excluded.
- Publications with potentially relevant content were retained for full-text assessment.

Phase 3: Full-Text Review and Quality Assessment

- The full text of the remaining publications was reviewed to determine their relevance to the research questions.

- Studies were assessed based on their research methodology, technical contribution, relevance to RAG security, quality of evaluation, and reliability of findings.
- Studies that did not provide sufficient relevance or research quality were excluded.

Phase 4: Final Selection and Organization

- The eligible publications were selected for detailed analysis and synthesis.
- Selected studies were organized according to their main research themes, including RAG and LLM fundamentals, prompt injection, corpus poisoning, privacy leakage, access control, retrieval manipulation, hallucination, and security frameworks.
- The selected literature was then analyzed to identify existing approaches, their limitations, and research gaps relevant to the proposed **RAGShield** framework.

The overall study selection process can be summarized as:

Identification → Deduplication → Title and Abstract Screening → Full-Text Review → Quality Assessment → Final Selection

2.1.7 Study Selection Results

The literature search and screening process resulted in a set of studies relevant to the scope of RAG security. Following the identification, screening, and eligibility assessment stages, 19 studies were considered eligible for inclusion. Due to the scope and depth of the analysis required, 11 of these studies were selected for detailed individual analysis in Section 2.4. The remaining eligible studies were considered during the broader thematic review and were used to support the understanding of the research area where relevant.

Table 2.2: Summary of the Study Selection Process

Stage	Number of records
Records identified through database searching	1292*
Duplicate records removed	3
Records screened by title and abstract	1289
Records excluded at title and abstract screening	1197
Full-text articles assessed for eligibility	19**
Studies selected for detailed individual analysis	11

** The initial database search returned 1292 records (IEEE Xplore: 227, ACM Digital Library: 12, Science Direct: 709, SpringerLink: 344). After removal of three duplicate records, 1289 unique records remained for screening.*

*** The detailed screening records should be checked against the original search log before final submission to ensure that the database and manual-identification branches reconcile correctly.*

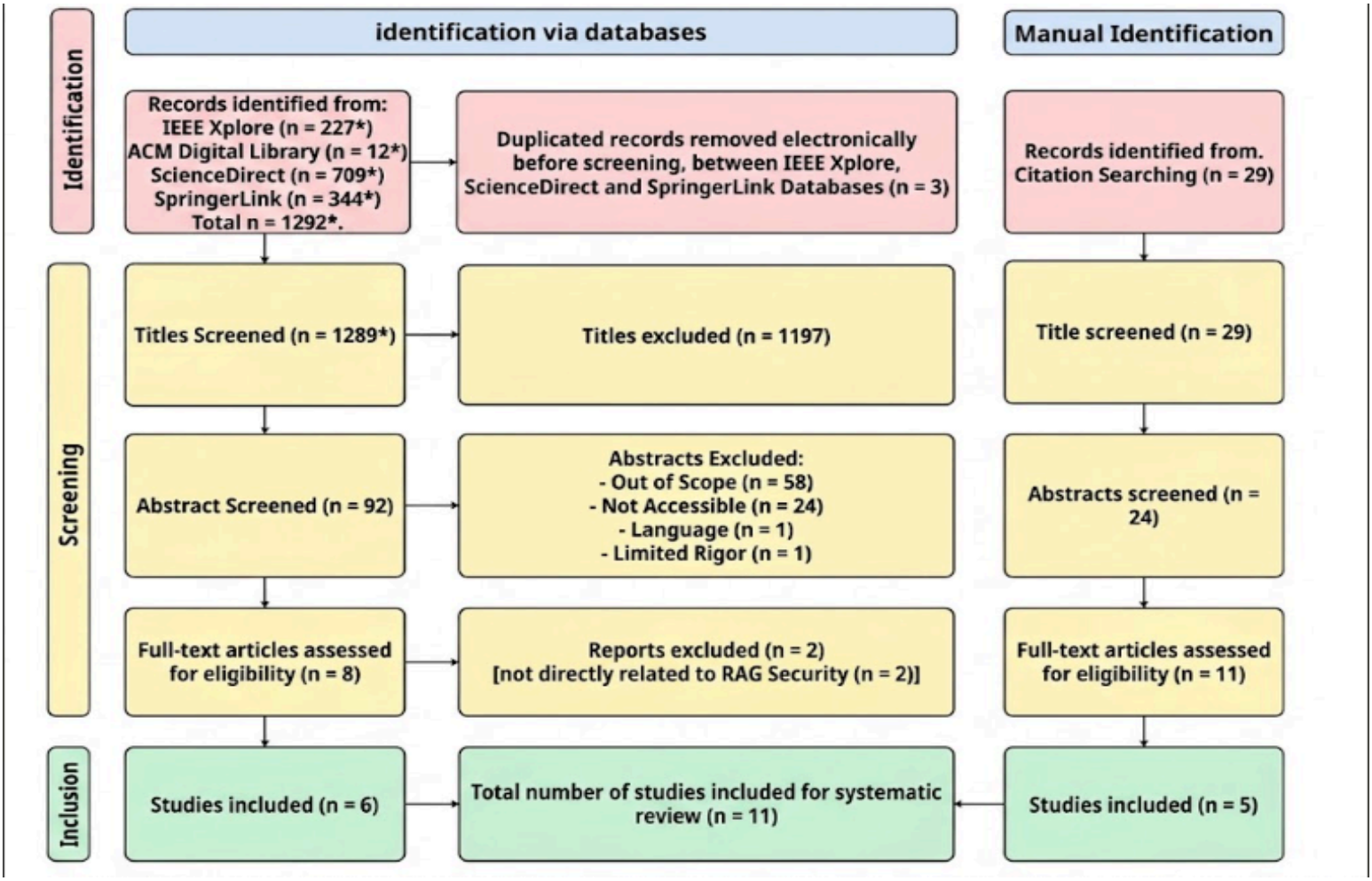

Fig. 2.1 PRISMA 2020 flow diagram documenting the systematic literature review process for RAG and LLM security studies.

2.1.8 Datasets for System Development and Evaluation

Based on the datasets identified throughout the literature review, a set of publicly available datasets is selected to evaluate RAGShield under different security scenarios. The selection is designed to cover both general-purpose RAG evaluation and the healthcare-oriented security requirements of the proposed framework.

For **prompt-injection detection**, the evaluation uses **AdvBench [21]**, **HackAPrompt [28]**, and the **ArabGuard-v1 dataset**, with the latter providing specific coverage of Arabic, Egyptian Arabic, and Franco-Arabic prompt-injection examples. These datasets are used to evaluate the ability of the input firewall to distinguish benign requests from malicious or adversarial prompts.

For **retrieval poisoning and knowledge-base security**, **Natural Questions (NQ) [22]**, **HotpotQA [23]**, and **MS MARCO [24]** are selected. These datasets are widely used in RAG and retrieval-poisoning research, including PoisonedRAG [6], and provide different question-answering and retrieval scenarios for evaluating whether malicious documents can influence the retrieved context and generated response.

For **healthcare-oriented RAG and privacy evaluation**, **ChatDoctor [25]**, **MedQA [40]**, and **HealthcareMagic-100k [42]** are considered. These datasets provide healthcare-related questions and conversational or medical knowledge that can be used to evaluate retrieval quality, sensitive-information handling, and the effectiveness of privacy controls within a healthcare RAG environment.

For **access-control evaluation**, the **Requirements Data Sets (User Stories) [32]** are considered because they contain user stories from multiple application domains, including healthcare and administrative systems. They provide a suitable source for evaluating retrieval and policy-related scenarios in which access decisions depend on the information associated with a user request.

The final experimental configuration will select the appropriate subset of these datasets according to the security layer being evaluated. This prevents a single dataset from being used as a proxy for all security properties and allows each RAGShield component to be evaluated against a threat model relevant to its intended function.

2.3 Security Threats Reported in the Reviewed Literature

Before the individual studies are analysed, this section consolidates the threat categories that recur across the selected literature. The categories are presented here rather than as part of

the technical background because they are an outcome of the review: they describe what the reviewed studies collectively report as the principal security concerns affecting RAG systems. They also provide the organising structure for the analysis that follows, since the themes used in Section 2.4 correspond directly to these threat categories.

2.3.1 Prompt Injection Attacks

Prompt injection is a security threat [2] in which an attacker attempts to influence an LLM by inserting instructions that conflict with the intended behavior of the application. In RAG systems, such instructions may originate from user input or externally retrieved content.

Attack Mechanism

LLMs process system instructions, user queries, and retrieved content within a shared context. Since the model may not reliably distinguish between trusted instructions and untrusted content, attackers can embed malicious instructions to influence the generated response.

Prompt injection can therefore occur at both the **input layer** and the **retrieval layer**. Direct injection originates from user input, while indirect injection is embedded within external content that becomes part of the retrieved context.

Major Forms

1. **Direct Prompt Injection:** Malicious instructions are included in user input to modify model behavior or bypass restrictions.
2. **Indirect Prompt Injection:** Malicious instructions are embedded in external documents and influence the LLM when retrieved.
3. **Instruction Override:** The attacker attempts to make the model disregard its original instructions.
4. **Context Leakage:** The attack attempts to expose information available within the model's context.
5. **Obfuscated Injection:** Malicious instructions are concealed using encoding, Unicode manipulation, or similar techniques.

Security Impact and Detection

Prompt injection can affect the integrity and confidentiality of RAG systems by manipulating responses, bypassing restrictions, or causing unauthorized information disclosure. Indirect injection is particularly concerning because malicious instructions may persist in retrieved content.

Detection approaches include pattern-based methods, machine-learning classifiers, semantic analysis, and techniques for separating trusted instructions from untrusted content. No single method is sufficient for all attack variants.

2.3.2 Retrieval Poisoning (Corpus Poisoning)

Retrieval poisoning, also known as corpus or document poisoning, occurs when malicious, false, or misleading content is introduced into the knowledge base. When retrieved, this content can influence the context provided to the LLM and affect the generated response.

Unlike prompt injection, which primarily targets the model's interpretation of instructions, corpus poisoning targets the **knowledge base and its contents**.

Attack Mechanism

The attack generally involves three stages:

6. **Content Injection:** Malicious or modified content is introduced into the knowledge base.
7. **Indexing and Retrieval:** The content is embedded and indexed and may later be retrieved for relevant queries.
8. **Response Influence:** The LLM uses the poisoned content as context and may generate inaccurate or manipulated responses.

Common Forms

- **Content Manipulation:** Modifying legitimate information.
- **Malicious Document Injection:** Adding attacker-controlled documents.
- **Authority Impersonation:** Presenting malicious content as coming from a trusted source.
- **Conflicting Information:** Introducing content that contradicts reliable sources.
- **Supply Chain Poisoning:** Introducing compromised content through external sources.

Security Impact and Detection

Corpus poisoning primarily threatens the **integrity and reliability** of RAG systems. Its effects may persist across multiple queries until the poisoned content is removed.

Detection and mitigation approaches include source verification, provenance tracking, integrity checking, cross-source validation, anomaly detection, and document trust scoring.

2.3.3 Privacy and PII/PHI Leakage

Privacy leakage occurs when sensitive information is unintentionally exposed through retrieved documents, model context, generated responses, or system metadata. This risk is particularly important when RAG systems process **Personally Identifiable Information (PII)** or **Protected Health Information (PHI)**.

Types of Sensitive Information

PII may include identifying and financial information, while PHI includes medical diagnoses, medical histories, medications, laboratory results, and treatment records. RAG systems may also contain other confidential information such as credentials and proprietary documents.

Privacy Threats

9. **Direct Output Leakage:** Sensitive information from retrieved documents appears in generated responses.
10. **Prompt-Based Extraction:** Attackers construct queries to obtain sensitive information.
11. **Embedding Leakage:** Compromised embedding stores may expose information represented within embeddings.
12. **Inference and Aggregation:** Multiple responses may be combined to infer sensitive information.
13. **Metadata Leakage:** Document metadata or retrieval information may reveal protected information.

Healthcare Privacy Concerns

Privacy protection is particularly important in healthcare RAG systems because medical knowledge bases may contain sensitive patient information. Unauthorized disclosure of PII or PHI can lead to serious privacy and compliance risks.

Detection and Mitigation

Common approaches include PII/PHI detection, de-identification, access control, encryption, privacy-preserving techniques, audit logging, and output monitoring.

2.3.4 Unauthorized Access and Broken Access Control

Unauthorized access occurs when users can retrieve information beyond their assigned permissions. This violates the **principle of least privilege**, which requires users to access only the information necessary for their authorized tasks.

Access Control in RAG Systems

Access control must be enforced during retrieval, not only at the application level. Common approaches include:

14. **Role-Based Access Control (RBAC):** Permissions are assigned according to predefined user roles.
15. **Attribute-Based Access Control (ABAC):** Access decisions consider user, resource, and contextual attributes.
16. **Context-Aware Access Control:** Factors such as device, location, time, or authentication status are considered.

Common Vulnerabilities

- **Document-Level Authorization Failure:** Retrieved documents are not filtered according to user permissions.
- **Privilege Escalation:** Users gain access beyond their assigned privileges.
- **Lateral Access:** A compromised account is used to access additional information.
- **Metadata Leakage:** Information about restricted documents is exposed.

These vulnerabilities are particularly critical in healthcare systems, where different users may have different access permissions.

Detection and Mitigation

Protection requires authorization checks before retrieved content is provided to the LLM. Additional measures include least-privilege policies, access auditing, monitoring, anomaly detection, and detailed logging.

2.3.5 Retrieval Manipulation

Retrieval manipulation occurs when an attacker influences which documents are selected or how they are ranked for a query. Since retrieved documents form the context provided to the LLM, manipulating retrieval can directly affect the generated response.

Attack Mechanism

RAG systems commonly use semantic similarity between query and document embeddings. Attackers may exploit this process by creating or modifying content to achieve high relevance scores, causing misleading content to be prioritized over legitimate information.

Major Forms

17. **Semantic Manipulation:** Modifying content to increase similarity to targeted queries.
18. **Adversarial Embeddings:** Crafting documents with embeddings designed to match specific queries.
19. **Ranking Manipulation:** Influencing the ranking of retrieved documents.
20. **Content Suppression:** Reducing the likelihood that legitimate documents are retrieved.

Security Impact and Detection

Retrieval manipulation threatens the **integrity and reliability** of RAG systems. In healthcare applications, prioritizing inaccurate information or suppressing reliable sources may affect the quality of generated responses.

Detection approaches include embedding and ranking analysis, retrieval consistency checks, source verification, and relevance or factuality assessment.

2.3.6 Hallucination and Untrusted Context

Hallucination refers to the generation of plausible but factually incorrect, fabricated, or unsupported information. Although RAG can improve grounding by providing external context, it does not completely eliminate hallucination. The reliability of the response depends on the quality and trustworthiness of the retrieved information.

Causes in RAG Systems

21. **Incomplete or Irrelevant Context:** Retrieved information may be insufficient or only partially relevant.
22. **Contradictory Information:** Retrieved sources may contain conflicting information.
23. **Generation Beyond Evidence:** The LLM may generate information that is not supported by the retrieved context.
24. **Untrusted or Outdated Sources:** Retrieved documents may contain inaccurate or outdated information.

Security and Reliability Impact

Hallucination and untrusted context affect the **integrity, factuality, and reliability** of RAG responses. This is particularly important in healthcare, where unsupported information may negatively affect decision-making and patient safety.

Detection and Mitigation

Common approaches include:

- **Grounding Verification:** Checking whether claims are supported by retrieved context.
- **Source Attribution:** Linking claims to their supporting sources.
- **Fact and Consistency Checking:** Detecting unsupported or contradictory claims.
- **Confidence Estimation:** Identifying cases where available evidence is insufficient.
- **Constrained Generation:** Encouraging the LLM to rely on retrieved evidence.
- **Retrieval Quality Assessment:** Evaluating the relevance and reliability of retrieved documents.

2.4 Analysis of the Selected Studies

This section analyses the studies selected through the process described in Section 2.2. The studies are grouped by security themes rather than presented as an undifferentiated list, and the themes correspond to the threat categories consolidated in Section 2.3: prompt injection, poisoning and knowledge-base attacks, privacy, access control, and retrieval security.

Each study is analysed using a consistent template covering its authors and year of publication, the method it proposes, the dataset or experimental setup it uses, the metrics by which it is evaluated, its main results, and its limitations. Where an item cannot be confirmed from the sources currently available for this review, it is recorded as unavailable rather than estimated. Several studies contribute to more than one theme; in such cases the full analysis is given at the study's first appearance and the later theme refers back to it. Each theme concludes with a discussion that draws together what the studies collectively establish, which approaches are effective, which limitations remain, and what gap follows for the present research.

2.4.1 Prompt Injection

Prompt injection is the threat that arises when an attacker embeds instructions in text that the language model treats as part of its operative context. Because a RAG system concatenates a user query with retrieved passages before generation, injected instructions may enter either through the user input or through the knowledge base. The studies in this theme address detection at the input stage.

ArabGuard

Authors and Year. Doaa Karem and Salwa Osman published the ArabGuard study in February 2026 [2].

Proposed Method. ArabGuard proposes a language-specific prompt injection detection framework designed for Arabic input, with support for Egyptian Arabic dialect and Franco-Arabic variants. The framework combines an Arabic prompt injection dataset with a machine-learning-based classifier that analyzes incoming prompts and classifies them as either benign or malicious before they reach the language model.

Dataset / Experimental Setup. The study introduces an Arabic prompt injection dataset containing **2,322 prompts**, covering both benign and malicious examples. The dataset is designed to represent Arabic-language prompt injection scenarios, including Modern Standard Arabic, Egyptian Arabic, and Franco-Arabic inputs. The proposed machine-learning classifier is trained and evaluated using this dataset to assess its ability to distinguish malicious prompts from legitimate user inputs.

Evaluation Metrics. ArabGuard is evaluated using precision, recall, F1-score, and false positive rate (FPR). The reported results are **93.5% precision, 90.5% recall, 92.0% F1-score, and 7.5% false positive rate**. The reported evaluation also indicates that the normalization pipeline reduces the false positive rate by approximately **3.7%** compared with analysis of raw text.

Main Results. ArabGuard demonstrates strong performance in detecting prompt injection and jailbreaking attacks in Arabic-language inputs, particularly for Egyptian Arabic and Franco-Arabic variants. The model achieves a precision of **93.5%**, recall of **90.5%**, and F1-score of **92.0%**, while maintaining a false positive rate of **7.5%**. The results indicate that incorporating language-specific normalization and detection mechanisms can improve the stability of prompt-injection detection against linguistic variation and obfuscated inputs. The updated model is trained on the **ArabGuard-v1 dataset containing 2,322 samples** and incorporates an 11-stage normalization pipeline designed to improve detection against obfuscated payloads.

Limitations. Protection is concentrated at the input stage. An attack that reaches the model through a retrieved document rather than through the user prompt is outside the scope of the approach, as is any threat arising after the input has been accepted, including poisoning, unauthorised retrieval, privacy leakage, and unsafe output. The approach is also primarily designed around Arabic-language inputs, particularly Egyptian Arabic and Franco-Arabic, and

therefore does not by itself provide a complete security solution for multilingual RAG deployments.

Discussion: Prompt Injection

The literature in this theme establishes that input-stage classification is an effective and practical control, and that its effectiveness is bounded by the linguistic coverage of the data on which the detector was trained. ArabGuard demonstrates the value of language-specific detection by explicitly addressing Egyptian Arabic and Franco-Arabic, linguistic forms that may not be adequately represented in predominantly English-oriented security datasets. Its reported F1-score of 92.0% and false positive rate of 7.5% indicate that such specialization can provide a practical detection layer while limiting unnecessary rejection of legitimate inputs.

What remains unaddressed is the scope of the control rather than its quality. Input filtering assumes that the untrusted text is the text the user supplied. In a RAG system, this assumption does not hold because retrieved passages are also incorporated into the model context and can carry malicious instructions. A system protected only at the input stage therefore remains exposed to indirect prompt injection delivered through the knowledge base, a threat category that Section 2.3.2 identifies and that the poisoning literature analysed in the next theme demonstrates in practice.

Resulting gap. Input-stage detection is necessary but not sufficient. Coordinating input filtering with controls over what is admitted to and retrieved from the knowledge base is not addressed by the studies in this theme, and contributes directly to **Gap 1 and Gap 2 in Section 2.6**.

2.4.2 Poisoning and Knowledge-Base Attacks

Poisoning attacks target the integrity of the retrieval corpus. Because a RAG system grounds its answers in retrieved documents, an attacker who can insert or modify documents can influence generated responses without any access to the model itself. The studies in this theme comprise one that establishes the threat and one that proposes a defence against it.

PoisonedRAG

Authors and Year. W. Zou, R. Geng, B. Wang, and J. Jia, 2024 (preprint); published in the proceedings of the 34th USENIX Security Symposium, 2025 [3].

Proposed Method. A knowledge corruption attack against RAG. The work formulates the injection of a small number of crafted malicious passages into the knowledge database so that

an attacker-chosen target question elicits an attacker-chosen target answer. The attack is constructed to satisfy two conditions simultaneously: a retrieval condition, ensuring the malicious passage is retrieved for the target question, and a generation condition, ensuring the retrieved passage leads the model to produce the intended answer.

Dataset / Experimental Setup. The study evaluates PoisonedRAG on three established question-answering datasets: **Natural Questions (NQ), HotpotQA, and MS-MARCO**. The corresponding knowledge databases are used as the RAG corpora. Three retrievers are evaluated: **Contriever, Contriever-ms, and ANCE**, with multiple language models including PaLM 2, GPT-3.5, GPT-4, LLaMA-2, and Vicuna variants. The experiments consider both black-box and white-box attacker settings.

Evaluation Metrics. The primary metrics are **Attack Success Rate (ASR)**, measuring whether the target answer is successfully induced, and **F1-score**, measuring the quality of the retrieved malicious content with respect to the target. The study also evaluates the number of malicious passages required to achieve a successful attack.

Main Results. PoisonedRAG demonstrates that a small number of malicious passages can effectively control RAG outputs. The authors report an **ASR of up to 90% when injecting five malicious texts per target question into a knowledge database containing millions of texts**. Across the evaluated datasets and retrievers, the attack achieves very high success rates; for example, with the default retriever settings, black-box PoisonedRAG reaches ASRs of **0.97 on NQ, 0.99 on HotpotQA, and 0.91 on MS-MARCO**.

Limitations. The contribution is an attack rather than a defence. The work characterises the vulnerability and motivates mitigation but does not propose an integrated protection mechanism, and it does not address how a deployed system should verify document provenance or integrity at ingestion time. Its threat model concerns corpus content and does not extend to authorisation, privacy, or output-stage controls.

TrustRAG

Authors and Year. H. Zhou, K.-H. Lee, Z. Zhan, Y. Chen, Z. Li, Z. Wang, H. Haddadi, and E. Yilmaz, 2025 [4].

Proposed Method. A defence framework that filters malicious and irrelevant content before it is used for generation, using a two-stage mechanism. The first stage applies K-means clustering to identify suspicious retrieved passages based on their embedding distributions, while ROUGE-L is used to preserve clean content in single-injection scenarios. The second

stage uses the language model's internal knowledge to identify conflicts between retrieved evidence and internally generated knowledge and performs self-assessment to select the more reliable information. The framework is training-free and designed to operate with different language models.

Dataset / Experimental Setup. TrustRAG is evaluated on **HotpotQA, Natural Questions (NQ), and MS-MARCO** under corpus-poisoning conditions. The experiments consider multiple language models, including **Mistral-Nemo-12B and Llama-3.1-8B**, and compare TrustRAG against Vanilla RAG, RobustRAG, InstructRAG, and ASTUTE RAG under prompt-injection and PoisonedRAG attacks.

Evaluation Metrics. The primary metrics are **Answer Accuracy (ACC)** and **Attack Success Rate (ASR)**. ACC measures the correctness of the generated response, while ASR measures the proportion of target questions for which the attacker successfully induces the desired malicious answer. The study also considers retrieval-related performance and efficiency.

Main Results. TrustRAG reports that its two-stage defence can reduce attack success while preserving answer quality. The authors report reductions in ASR of **up to 80%** and improvements in response accuracy of **up to 30%** across different models and datasets. In the reported experiments, the complete TrustRAG pipeline substantially outperforms its Stage-1-only variant and competing defences. For example, with Mistral-Nemo-12B on HotpotQA, TrustRAG Stage 1&2 achieves **77% ACC and 9% ASR**, compared with **43% ACC and 49% ASR** for Vanilla RAG under the prompt-injection attack. On NQ, it achieves **74% ACC and 13% ASR**, while on MS-MARCO it achieves **81% ACC and 9% ASR**.

Limitations. The mechanism operates on the content of retrieved passages at query time and therefore treats poisoning as a filtering problem rather than as a document-lifecycle problem. It does not verify that an approved document has remained unmodified since ingestion, and it does not control what enters the corpus in the first place. Its self-assessment stage also depends on the judgement of the same class of model that the attack targets. The framework does not address authorisation, sensitive-data disclosure, or output validation.

2.4.3 Privacy

The Good and The Bad: Exploring Privacy Issues in Retrieval-Augmented Generation

Authors and Year. S. Zeng, J. Zhang, P. He, Y. Xing, Y. Liu, H. Xu, J. Ren, S. Wang, D. Yin, Y. Chang, and J. Tang, 2024, published in Findings of the Association for Computational Linguistics: ACL 2024 [6].

Proposed Method. An empirical study of privacy risk in RAG, approached from the attack perspective. The work develops targeted and untargeted extraction attacks that induce RAG systems to disclose content held in private retrieval databases. It also investigates whether retrieval augmentation affects leakage of the language model's own training data.

Dataset / Experimental Setup. The experiments use several retrieval datasets, including **ChatDoctor, Enron-Mail, and WikiText-103**, together with W3C email data in the privacy experiments. The official implementation provides preprocessing and evaluation scripts for constructing vector databases from these datasets and evaluating the resulting RAG systems.

Evaluation Metrics. The study evaluates privacy leakage primarily by measuring the **amount of private information successfully extracted**. For training-data reconstruction, the authors use **ROUGE-L similarity**, considering an extraction successful when the similarity score exceeds **0.5**. The targeted extraction experiments also report the number of successfully extracted private records or fields.

Main Results. The study demonstrates that private retrieval databases can be extracted through carefully constructed prompts, establishing a privacy risk specific to RAG. At the same time, the experiments show that adding retrieval data can reduce leakage from the language model's own training data. In the reported targeted-attack experiments, RAG configurations substantially reduced the number of training-data items extracted compared with direct attacks against the language model; for example, the reported results include only **2, 1, and 15** extracted items for the ChatDoctor retrieval setting across the evaluated private fields, compared with substantially larger counts for the non-RAG settings.

Limitations. The contribution is an empirical characterisation of the risk rather than a deployable defence. The work motivates the need for dedicated privacy controls in RAG but does not specify a complete mechanism for detecting or masking sensitive content, nor does it integrate privacy protection with authorisation or output filtering.

SecureRAG

Authors and Year. A. Bassit and V. Boddeti, 2025 [5].

Proposed Method. An end-to-end secure RAG framework that decouples retrieval into secure search and secure document fetching. Fully Homomorphic Encryption (FHE) is used to perform similarity search over encrypted vectors, while Attribute-Based Encryption (ABE) enforces fine-grained access control over which documents a requester may decrypt. The

framework supports dynamic database updates and adaptive access policies and is designed to mitigate prompt injection, data extraction, and embedding inversion attacks.

Dataset / Experimental Setup. SecureRAG is evaluated using **PubMedQA and CovidQA-RAG** for biomedical applications, **TechQA** for customer support, and **FinQA and TAT-QA** for financial applications. The experiments use **ModernBERT Embed** as the retriever, **Llama-2-7B** as the FHE-compatible generator, and **Llama-3.1-8B** as the LLM judge, without fine-tuning the models.

Evaluation Metrics. The study measures retrieval performance using **Rank@k** and **Context Precision**, with the latter evaluated by an LLM judge according to the relevance of retrieved documents to the question and generated response. Runtime is additionally measured for encrypted search, secure document fetching, and encrypted LLM inference.

Main Results. SecureRAG maintains retrieval quality under encryption with minimal degradation. The reported context-precision results are close to the cleartext baseline, with values approaching **0.999 for k=5 and k=10** across most evaluated datasets and embedding dimensions. The authors also report that encrypted search can be performed with low overhead at smaller database sizes and that the overall framework adds approximately **0.05 seconds of overhead** under the reported configuration.

Limitations. The approach depends on homomorphic encryption operators and on integration with FHE-compatible language models, which constrains the choice of generation model and complicates deployment on top of existing commercial LLM services. It requires key-management and policy infrastructure that many clinical environments may not already operate. Confidentiality is enforced at the document level, so the framework does not address sensitive content that appears inside a document a user is legitimately entitled to retrieve. It also does not provide a comprehensive solution for corpus integrity, output validation, or lifecycle-wide auditing.

2.4.4 Retrieval Security and Retrieval Quality

CRAG (Corrective Retrieval-Augmented Generation)

Authors and Year. S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, 2024 [7].

Proposed Method. A corrective mechanism that improves robustness when retrieval performs poorly. A lightweight retrieval evaluator assesses the quality of retrieved passages and produces a confidence score that triggers different knowledge-retrieval actions. When local retrieval is insufficient, large-scale web search is used to supplement the retrieved information.

A decompose-then-recompose mechanism then extracts key information from retrieved passages while discarding irrelevant content. The method is designed as a plug-and-play component for existing RAG pipelines.

Dataset / Experimental Setup. CRAG is evaluated on four datasets covering short- and long-form generation tasks: **PopQA, PubQA, Bio, and ARC-Challenge**. The released implementation provides the corresponding evaluation data and retrieval-evaluation training data used by the retrieval evaluator.

Evaluation Metrics. The experiments evaluate the quality of generated answers using task-appropriate generation metrics, including **Exact Match (EM) and token-level F1**, together with comparisons against existing RAG and self-correcting approaches.

Main Results. The authors report that CRAG significantly improves the performance of RAG-based approaches across the four evaluated datasets and that the method can be coupled with different RAG architectures. The main contribution is improved robustness when retrieved documents are insufficient, incorrect, or noisy rather than protection against malicious retrieval content.

Limitations. The objective is generation robustness rather than security: the retrieval evaluator assesses whether passages are relevant and sufficient, not whether they are adversarial, authentic, or authorised. The use of web search as a fallback also expands the trust boundary by admitting content from an uncontrolled external source, which can introduce additional security risks in a clinical setting. The method does not address poisoning, authorisation, privacy leakage, or output validation.

MiRAG (Multi-Level Information Retrieval-Augmented Generation)

Authors and Year. O. Adjali, O. Ferret, S. Ghannay, and H. Le Borgne, 2024, published in the Proceedings of EMNLP 2024 [8].

Proposed Method. A multi-level retrieval approach that improves answer generation through entity retrieval followed by query expansion. Entities retrieved at a coarse level are added to the question before a second, finer-grained passage-level retrieval is performed. A joint training objective conditions answer generation on both entity-level and passage-level retrieval.

Dataset / Experimental Setup. The method is evaluated on the **VIQuAE knowledge-based visual question answering benchmark**, which combines visual question answering with textual and visual knowledge retrieval. The experiments investigate whether entity-level

retrieval followed by passage-level retrieval can surface more relevant knowledge for answer generation.

Evaluation Metrics. The evaluation focuses on **retrieval effectiveness and answer-generation performance** on the VIQuAE benchmark, using the benchmark's answer-generation and retrieval evaluation measures.

Main Results. MiRAG reports **state-of-the-art performance on the VIQuAE benchmark at the time of publication** and demonstrates that multi-level retrieval can retrieve more genuinely relevant knowledge for answer generation.

Limitations. The work is primarily a retrieval-effectiveness contribution and makes no security claim. It is evaluated in a knowledge-based visual question-answering setting rather than in a text-only enterprise or clinical RAG deployment. Consequently, its findings establish the importance of retrieval architecture for context quality but do not establish protection against poisoning, prompt injection, unauthorised retrieval, or privacy leakage.

2.4.5 Summary of the Analysed Studies

The analysis above shows that effective mechanisms exist for individual stages of the RAG lifecycle. ArabGuard [2] addresses prompt injection at the input stage; PoisonedRAG [3] establishes the practicality of knowledge-base poisoning and TrustRAG [4] defends against it at retrieval time; Zeng et al. [6] demonstrate extraction of the retrieval database and SecureRAG [5] provides cryptographic confidentiality together with attribute-based authorisation; CRAG [7] and MiRAG [8] improve the reliability and effectiveness of retrieval itself.

The studies differ substantially in objective as well as in scope. Two are attack studies, two are security mechanisms, and two are retrieval-effectiveness contributions. None of them provides protection spanning the full lifecycle, none coordinates controls across stages, and none reports lifecycle-wide monitoring or auditing. The comparative analysis presented in the following section makes this distribution of coverage explicit.

2.5 Comparative Analysis of the Selected Studies

The reviewed studies address different security, privacy, retrieval, and reliability challenges within RAG systems. However, their objectives and security coverage vary considerably. To

identify the strengths and limitations of existing approaches, the selected studies are compared according to the major security capabilities considered in this project.

The comparison focuses on input security, knowledge-base integrity and poisoning protection, privacy protection, retrieval security, access control, output validation, and monitoring. A distinction is made between studies that provide an actual defense mechanism and studies that primarily analyze or demonstrate a particular vulnerability.

Table 2.3: Security Coverage of the Selected Studies Compared with the Proposed RAGShield Framework

Study	Input Security	Poisoning / Data Integrity	Privacy Protection	Retrieval Security	Access Control	Output Validation	Monitoring
ArabGuard [2]	✓	—	—	—	—	—	—
SecureRAG [5]	—	—	✓	✓	✓	—	—
MiRAG [8]	—	—	—	✓	—	—	—
Zeng et al. [6]	—	—	Analysis	—	—	—	—
PoisonedRAG [3]	—	Analysis	Analysis	Analysis	—	—	—
TrustRAG [4]	—	—	—	✓	—	—	—
CRAG [7]	—	—	—	✓	—	✓	—
RAGShield (proposed)	**Proposed**	**Proposed**	**Proposed**	**Proposed**	**Proposed**	**Proposed**	**Proposed**

Note. *A tick indicates a capability that the study demonstrates and reports experimentally. "Analysis" indicates that the study investigates or demonstrates a threat rather than providing a defence mechanism for it. "Proposed" indicates a capability that RAGShield is designed to provide but that has not yet been experimentally demonstrated; the entries in the RAGShield row are therefore design commitments to be evaluated, not results. This distinction is maintained throughout the report.*

The comparison reveals several important observations.

First, **input-level security** is directly addressed by ArabGuard [2] through prompt injection detection. The study provides a specialized mechanism for Arabic and Egyptian dialect

prompts, but its protection is primarily concentrated on the input stage. Consequently, threats introduced after the input has been accepted are outside its main security scope.

Second, **privacy and cryptographic protection** receive significant attention in SecureRAG [5]. The framework demonstrates how encryption and attribute-based access mechanisms can be incorporated into secure retrieval. However, the use of advanced cryptographic techniques may introduce computational overhead, increased retrieval latency, and additional deployment complexity.

Third, several studies focus primarily on **retrieval quality, trust, or reliability**. MiRAG [8] improves the retrieval process through multi-level retrieval and entity-based techniques, TrustRAG [4] introduces trust-aware document selection, and CRAG [7] evaluates retrieved information and applies corrective retrieval when retrieval quality is insufficient. These contributions improve the reliability of retrieval but do not provide comprehensive protection against the full range of security threats considered in this project.

Fourth, **poisoning and privacy attacks** are demonstrated effectively in PoisonedRAG [3] and Zeng et al. [6], respectively. These studies provide important evidence of the practical risks associated with vulnerable knowledge bases and retrieval databases. However, their primary focus is threat analysis rather than an integrated security architecture capable of continuously preventing and monitoring such attacks.

Finally, **monitoring and lifecycle-wide security coverage** are not explicitly addressed by the reviewed studies. Most of the existing approaches focus on one specific security layer or capability. For example, a system may protect the input against prompt injection while providing no mechanism to verify retrieved documents, enforce fine-grained authorization, or validate the final response.

From the perspective of the complete RAG lifecycle, the reviewed approaches can therefore be summarized as follows:

Document Ingestion
↓
Embedding / Vector Storage
↓
Retrieval
↓
Context Construction
↓
LLM Generation
↓
Output

Existing studies primarily secure individual stages or individual security properties within this lifecycle. This fragmentation represents an important characteristic of the current research landscape and provides the foundation for identifying the research gaps discussed in the next section.

2.6 Research Gaps

The analysis in Section 2.4 and the comparison in Table 2.3 show that substantial progress has been made on individual security and reliability problems in Retrieval-Augmented Generation. The theme discussions also show that the limitations of the reviewed work are consistent rather than incidental: they recur across studies with different objectives and different methods. Consolidating those recurring limitations yields four research gaps, each of which follows from evidence in the reviewed literature rather than from assertion.

2.6.1 Gap 1: Fragmented Security Coverage Across the RAG Lifecycle

Existing research addresses individual stages of the RAG lifecycle or individual attack types rather than providing protection across the complete pipeline. The evidence for this is visible directly in Table 2.3: no reviewed study occupies more than three of the seven capability columns, and five of the seven occupy only one. ArabGuard [2] secures the input stage and explicitly does not extend to content arriving through retrieval. TrustRAG [4] secures the retrieval stage against poisoned content but neither validates documents at ingestion nor examines the generated response. SecureRAG [5] secures storage and retrieval cryptographically but does not address adversarial content within documents a user is entitled to read. CRAG [7] evaluates the sufficiency of retrieved passages but not their authenticity or authorisation status.

The consequence is structural rather than a matter of degree. A pipeline assembled from these defences would still admit a poisoned document at ingestion, would still surface protected health information contained within an authorised document, and would still deliver a response derived from manipulated context, because no reviewed approach places a control at those points. The gap is therefore the absence of an architecture in which every stage of the

lifecycle, from document ingestion through to response delivery, is mediated by a security control.

2.6.2 Gap 2: Limited Coordination Between Security Controls

Beyond the question of coverage, the reviewed studies provide controls that operate independently of one another and do not exchange the information each would need in order to make a correct decision. The discussion in Section 2.4.5 identified the clearest instance: a retrieved passage simultaneously has a relevance score, a trust status, an integrity status, an authorisation status, and a sensitive-content status, yet each reviewed approach conditions its decision on exactly one of these. TrustRAG [4] evaluates trust without regard to authorisation; SecureRAG [5] evaluates authorisation without regard to whether the authorised document has been tampered with or contains adversarial instructions; CRAG [7] evaluates sufficiency without regard to either.

The same absence of coordination appears between stages. Section 2.4.1 established that input-stage filtering assumes the untrusted text is the text the user supplied, an assumption that fails once retrieved passages are concatenated into the model context; a coordinated design would apply the same injection analysis to retrieved content that it applies to user input. Section 2.4.3 established that document-granular confidentiality does not express field-level sensitivity, which requires the privacy control and the authorisation control to inform one another rather than operate in sequence. The gap is therefore not the absence of individual controls but the absence of a mechanism by which their decisions are combined.

2.6.3 Gap 3: Limited Evaluation Under Compound Attacks

The reviewed studies evaluate threats in isolation. PoisonedRAG [3] evaluates corpus poisoning against a system with no other adversarial pressure applied; Zeng et al. [6] evaluate retrieval-database extraction against a system that is not simultaneously being poisoned; TrustRAG [4] evaluates its defence against poisoned passages that do not also attempt to escalate privilege or exfiltrate protected data.

This is a significant omission because the mechanisms these studies describe compose naturally. A passage crafted to satisfy PoisonedRAG's retrieval condition can equally carry an injected instruction, since both are properties of the same passage text, and the combination would traverse an input filter untouched because the malicious text never appears in the user prompt. Nothing in the reviewed literature establishes how the defences behave under such combinations, whether a control that is effective against a single-stage attack remains effective when an earlier control has already been evaded, or whether independently validated controls

compose without interfering with one another. Evaluation against multi-stage and combined threat scenarios is therefore required before lifecycle-wide claims can be substantiated.

2.6.4 Gap 4: Insufficient Analysis of the Security–Utility–Performance Trade-off

Security mechanisms in RAG systems act directly on the retrieval and generation path, and therefore have consequences for retrieval quality, answer accuracy, latency, and computational cost. The reviewed literature addresses this unevenly. SecureRAG [5] is the only selected study that reports security effectiveness and cost together, reporting retrieval latency alongside ranking accuracy and context precision, and its results indicate that the trade-off is more favourable than commonly assumed. Elsewhere the two are separated: the security studies report protection without reporting the effect of their filtering on answer quality for benign queries, while the retrieval studies [7], [8] report quality improvements without evaluating the security implications of the designs that produce them.

Section 2.4.5 identified a concrete instance in which the two are in tension: CRAG's use of web search as a fallback improves answer quality precisely by admitting content from an uncontrolled source, which enlarges the attack surface. Conversely, aggressive filtering at retrieval time protects against poisoning but may discard passages a clinician legitimately needs, and cryptographic protection constrains the choice of generation model. Because a security framework that degrades retrieval quality or response time beyond an acceptable threshold will not be adopted in a clinical setting regardless of its protective properties, a security framework for healthcare RAG must be evaluated on both dimensions together rather than on protection alone.

2.6.5 Summary of the Identified Gaps

The four gaps are related but distinct. Gap 1 concerns which stages of the lifecycle are protected at all; Gap 2 concerns whether the controls at those stages act on a shared view of the request; Gap 3 concerns whether protection has been validated under realistic, combined threat conditions; and Gap 4 concerns whether protection can be provided at a cost the application domain can absorb. Together they define the requirements that the proposed RAGShield framework is designed to satisfy, and they are the basis for the system requirements specified in Chapter Three.

2.7 RAGShield Positioning and Contribution

The research gaps identified in the reviewed literature provide the motivation for the proposed RAGShield framework. The analysis shows that existing studies provide valuable solutions for specific RAG security and reliability challenges, but their security mechanisms are generally distributed across individual stages or isolated threat categories.

RAGShield is positioned as a multi-layer security framework designed to coordinate protection across the major stages of the RAG lifecycle. Rather than addressing a single threat, the framework integrates multiple security mechanisms intended to protect the system from the initial processing of user input through document retrieval and final response generation.

The proposed framework is designed around the following security layers.

2.7.1 Input Security Layer

The input security layer is responsible for analyzing user-provided queries before they enter the RAG pipeline. The framework combines pattern-based analysis with machine-learning and language-model-based assessment to identify potentially malicious prompt injection attempts.

This layer extends the type of input-focused protection demonstrated by approaches such as ArabGuard while considering the need for additional security controls beyond the initial user input.

2.7.2 Secure Document Ingestion and Integrity

RAGShield incorporates security controls during the document ingestion process to reduce the risk of malicious or unauthorized content entering the knowledge base.

Documents can be subjected to validation and integrity verification before becoming available for retrieval. Cryptographic integrity mechanisms, such as HMAC-based verification, can be used to detect unauthorized modification of trusted documents.

This layer addresses a key limitation identified in approaches that analyze poisoning attacks without providing an integrated document-ingestion defense.

2.7.3 Privacy Protection

The proposed framework considers sensitive information protection as an independent security layer. Sensitive information may be detected and protected before it is exposed through retrieved context or generated responses.

Context-preserving masking can be used to replace sensitive values with controlled placeholders while maintaining sufficient contextual information for downstream processing. This approach provides an alternative to relying exclusively on computationally expensive cryptographic retrieval mechanisms.

For healthcare applications, this layer is particularly important because retrieved information may contain Personally Identifiable Information (PII) and Protected Health Information (PHI).

2.7.4 Retrieval Security and Trust Evaluation

RAGShield treats retrieval as an active security control point rather than a purely relevance-based operation. Retrieved documents can be evaluated using multiple signals, including relevance, trustworthiness, integrity status, and potential malicious characteristics.

A Retrieval Firewall is proposed to prevent untrusted or suspicious content from being passed directly to the generation stage. Trust evaluation can also be incorporated into retrieval decisions, complementing concepts explored by TrustRAG and retrieval-quality approaches such as CRAG and MiRAG.

2.7.5 Dynamic Access Control

The framework incorporates access control into the retrieval process so that authorization decisions can be applied before sensitive information becomes part of the LLM context.

RAGShield is designed to support Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC), allowing authorization decisions to depend on user roles and additional contextual attributes.

This approach aims to provide more flexible authorization than static permissions and to reduce the possibility of unauthorized information reaching the generation stage.

2.7.6 Output Validation

RAGShield introduces a final validation layer before the generated response is delivered to the user. The generated output can be analyzed for sensitive information, policy violations, and content that may have been influenced by untrusted or malicious retrieved context.

Output validation provides an additional security boundary because an attack may bypass earlier layers or produce an unsafe response despite successful input and retrieval processing.

2.7.7 Continuous Monitoring and Auditing

The proposed framework also considers security monitoring as part of the RAG lifecycle. Security-relevant events can be logged to support detection, investigation, and auditing.

Examples of auditable events include rejected prompts, suspicious documents, integrity failures, blocked retrieval attempts, authorization violations, sensitive-data detections, and output-validation failures.

This monitoring capability addresses a limitation observed across the reviewed studies, where security controls are generally evaluated as isolated mechanisms without an integrated lifecycle-oriented auditing layer.

2.7.8 Overall Positioning

The positioning of RAGShield relative to the reviewed approaches can be summarized as follows:

Table 2.4: Positioning of RAGShield Relative to the Selected Studies

Security Capability	ArabGuard [2]	SecureRAG [5]	PoisonedRAG [3]	TrustRAG [4]	CRAG [7]	RAGShield
Prompt Injection Protection	✓	—	—	—	—	**Proposed Coverage**
Document Integrity / Poisoning Defence	—	Partial	Analysis	Partial	—	**Proposed Coverage**
Privacy Protection	—	✓	—	—	—	**Proposed Coverage**
Retrieval Security	—	✓	Analysis	✓	✓	**Proposed Coverage**
Access Control	—	✓	—	—	—	**Proposed Coverage**
Output Validation	—	—	—	—	Partial	**Proposed Coverage**
Security Monitoring	—	—	—	—	—	**Proposed Coverage**
Multi-Layer Security	—	Partial	—	—	—	**Proposed Coverage**

Note. *Entries for the existing studies record capabilities those studies demonstrate and report. Entries for RAGShield record proposed coverage: they describe what the framework is designed to provide, and they are not experimental results. The evaluation of whether that coverage is achieved is presented in the evaluation chapter.*

The table illustrates the intended positioning of RAGShield as an integrated security framework rather than a solution dedicated to a single threat. The framework is designed to combine input protection, document integrity, privacy protection, retrieval security, dynamic authorisation, output validation, and security monitoring within a unified architecture, and the coordination between these controls is intended to address Gap 2 in the same way that their combined coverage addresses Gap 1.

The contribution of RAGShield therefore lies primarily in the integration and coordination of these security controls across the RAG lifecycle. The framework is intended to provide a practical security architecture suitable for sensitive environments while avoiding unnecessary dependence on computationally expensive mechanisms where lighter-weight controls can provide an appropriate security-performance trade-off.

2.8 Chapter Summary

This chapter presented a structured review of existing research on the security, privacy, reliability, and trustworthiness of Retrieval-Augmented Generation systems. It began by documenting the review methodology, including the research questions, the sources searched, the search strategy, the inclusion and exclusion criteria, the four-phase selection process, and the outcome of that process.

The threat categories reported across the selected literature were then consolidated, covering prompt injection, corpus poisoning, privacy and PII/PHI leakage, unauthorised access, retrieval manipulation, and untrusted context or hallucination. These categories provided the organising structure for the analysis of the selected studies, which were examined individually using a consistent template covering authorship, proposed method, experimental setup, evaluation metrics, main results, and limitations, and were grouped into five security themes. Each theme concluded with a discussion drawing together what the studies collectively establish and what they leave unresolved.

The analysis showed that ArabGuard [2] addresses prompt injection at the input stage; that PoisonedRAG [3] establishes the practicality of knowledge-base poisoning while TrustRAG [4] defends against it at retrieval time; that Zeng et al. [6] demonstrate extraction of the private retrieval database while SecureRAG [5] provides cryptographic confidentiality together with attribute-based authorisation; and that CRAG [7] and MiRAG [8] improve retrieval reliability and effectiveness without addressing security. The comparative analysis in Table 2.3 made the resulting distribution of coverage explicit: no reviewed study spans more than three of the seven capabilities considered, and none reports lifecycle-wide monitoring.

Four research gaps were derived from this analysis: fragmented security coverage across the RAG lifecycle, limited coordination between security controls, limited evaluation under compound attacks, and insufficient analysis of the security–utility–performance trade-off. RAGShield was then positioned against these gaps as a multi-layer architecture integrating input security, document integrity verification, privacy protection, retrieval security, dynamic access control, output validation, and continuous monitoring. As Table 2.4 records, this coverage is proposed rather than demonstrated, and the evaluation of whether it is achieved is reported in the evaluation chapter.

The findings of this literature review therefore establish the theoretical and practical motivation for the proposed RAGShield architecture. The following chapter presents the system requirements and design of the proposed framework, including its components, interactions, and security mechanisms.

CHAPTER THREE

System Analysis and Design

3.1 Introduction

This chapter presents the analysis and design of the proposed RAGShield framework. Chapter Two established that existing defences for Retrieval-Augmented Generation systems are effective but fragmented, each securing a single stage of the pipeline while leaving the remaining stages exposed. The purpose of the present chapter is to translate that finding, together with the objectives stated in Chapter One, into a concrete and implementable technical specification for an end-to-end, zero-trust security architecture.

The chapter opens with the overall architecture of the framework in Section 3.2. That section presents the system as a whole, identifying its major components, the order in which a request traverses them, and the trust boundary each security layer defends, so that the detailed models presented subsequently can be read against a single coherent picture of the system rather than assembled from them.

The design is expressed using the Unified Modeling Language (UML) together with structured data-modelling notation, so that both the behaviour and the structure of the system are captured. Use case modelling defines the functional boundary of the system and the responsibilities of each actor. Activity diagrams describe the sequential control flow of the principal operational processes. Sequence diagrams expose the layered message exchanges between internal components for the most security-critical scenarios. The domain class diagram and the entity-relationship diagram define the static structure of the system at the object and persistence levels respectively, and data flow diagrams at Levels 0 and 1 describe how information moves between the system, its users, and its external dependencies.

A defining characteristic of the design presented here is that security is treated as an architectural property rather than as an auxiliary feature. Four principles are therefore applied consistently across every model in this chapter. **Complete mediation** requires that every request be validated, authorised, and audited, irrespective of its origin or of any prior authorisation granted to the same user. **Least privilege** requires that each actor be granted only the minimum access necessary for the task at hand. The **fail-closed** principle requires that any request whose safety cannot be positively established be rejected rather than forwarded. Finally, **tamper-evident accountability** requires that every security-relevant action be recorded in a form whose subsequent modification can be detected. These principles are what distinguish the proposed design from a conventional RAG pipeline to which isolated controls have been added.

The remainder of the chapter is organised as follows. Section 3.3 specifies the functional and non-functional requirements of the framework. Section 3.4 presents the functional model

through the use case diagram and identifies the system actors. Section 3.5 describes the operational workflows using activity diagrams, and Section 3.6 details the dynamic interaction between components using sequence diagrams. Sections 3.7 and 3.8 define the structural and database designs, and Section 3.9 presents the data flow models. Section 3.10 cross-checks the design models against one another and records the points at which they diverge. Section 3.11 establishes traceability between the security layers proposed in Chapter Two, the design models presented here, and the requirements defined in Section 3.3. Section 3.12 summarises the chapter.

3.2 Overall RAGShield Architecture

Before the individual design models are presented, this section describes the architecture as a whole, so that the detailed diagrams that follow can be read against a single coherent picture of the system. The architecture realises the seven security layers proposed in Section 2.7 as an ordered sequence of controls through which every request must pass, and it is the structure that the use case, activity, sequence, class, entity-relationship, and data flow models each describe from a different perspective.

The organising principle is that the RAG lifecycle is treated as a chain of trust boundaries rather than as a data-processing pipeline with security attached to it. A request crosses a boundary when it moves from one component to the next, and at each boundary the request is evaluated afresh rather than being trusted on the strength of having passed an earlier check. This is the architectural expression of the complete-mediation and fail-closed principles stated in Section 3.1, and it is what distinguishes the design from a conventional RAG pipeline with isolated controls added to it.

3.2.1 End-to-End Request Flow

The path taken by a clinical query through the system is shown below. Each stage corresponds to one of the internal processes defined in the Level 1 data flow model in Section 3.9.2, and the component names used here are the names used consistently throughout the remainder of the chapter.

User (Clinical or Administrative Staff)

↓

Input Security Firewall and Risk Assessment (P1)

↓

Sensitive Data Detection and Masking (P2)

↓

Access Control and Query Authorisation (P5)

↓

Retrieval Firewall and Trust Evaluation (P6)

↓

Encrypted Vector Database (D1)

↓

Trusted Retrieved Context

↓

Secure Context Builder and LLM Interface (P7)

↓

External Large Language Model (E6)

↓

Output Firewall and Hallucination Check (P8)

↓

Controlled Unmasking, Audit and Response Delivery (P9)

↓

Response to User

3.2.2 Architectural Layers and Their Responsibilities

Table 3.1 relates each security layer proposed in Section 2.7 to the components that realise it and to the trust boundary it defends. The final column records the research gap from Section 2.6 that the layer is intended to address, so that the architecture can be read as a direct response to the literature analysis rather than as an independent design.

Table 3.1: Overall Architecture — Security Layers, Components and Purpose

Security Layer	Realising Component	Responsibility at This Boundary	Gap Addressed
Input security	Input Security Firewall and Risk Assessment (P1)	Evaluates every incoming query for adversarial or injected content and assigns a risk score before the query enters the pipeline; rejects the request where safety cannot be established.	Gap 1
Privacy protection	Sensitive Data Detection and Masking (P2); Masking Token Vault (D2)	Detects PII and PHI and replaces sensitive values with controlled placeholders, preserving context for downstream processing while withholding the underlying values.	Gap 1, Gap 2
Dynamic access control	Access Control and Query Authorisation (P5); Access Policies and Trust Stores (D3)	Evaluates the request against role- and attribute-based policy before any content is retrieved, so that unauthorised material never reaches the model context.	Gap 1, Gap 2
Retrieval security and trust evaluation	Retrieval Firewall and Trust Evaluation (P6); Encrypted Vector Database (D1)	Combines relevance with trust score, integrity status and authorisation status in a single admission decision over candidate passages, rather than ranking by similarity alone.	Gap 2
Secure document ingestion and integrity	Document Ingestion, Chunking and Embedding (P3); Vector Database Management (P4)	Validates documents at ingestion and maintains tamper-evident integrity state, so that modification of an approved document after ingestion is detectable.	Gap 1
Output validation	Output Firewall and Hallucination Check (P8)	Inspects the generated response for residual sensitive content, policy violations, and evidence of influence by untrusted context, providing a final boundary should an earlier control be evaded.	Gap 1, Gap 3
Continuous monitoring and auditing	Controlled Unmasking, Audit and Response Delivery (P9); Audit Log Store (D4)	Records every security-relevant decision taken along the request path in a tamper-evident hash chain, supporting detection, investigation and accountability across the lifecycle.	Gap 1, Gap 3

Note. *The layers are not independent filters applied in sequence. The trust evaluation performed at P6 consumes the integrity state maintained by P3 and P4 and the authorisation decision issued by P5, and the controlled unmasking performed at P9 depends on both the*

token mapping created by P2 and the authorisation decision issued by P5. This exchange of state between controls is the architectural response to Gap 2, and it is the property that the sequence diagrams in Section 3.6 are intended to make visible.

3.2.3 Separation of Persistent State

The architecture maintains four separate data stores: the Encrypted Vector Database (D1), the Masking Token Vault (D2), the Access Policies and Trust Stores (D3), and the Audit Log Store (D4). The separation is deliberate rather than incidental. Holding the masking token mapping apart from the indexed content ensures that compromise of the vector database does not by itself permit reidentification of the masked values it refers to, and holding the audit log in a tamper-evident hash chain ensures that an attacker who reaches the system cannot silently erase the record of having done so. The structure of these stores is specified in the entity-relationship model in Section 3.8.

With the overall architecture established, the remainder of this chapter specifies the requirements the system must satisfy and then presents each design model in turn. Every model that follows describes some aspect of the architecture set out in this section, and Section 3.10 establishes explicit traceability between the layers described here, the models that realise them, and the requirements they satisfy.

3.3 System Requirements

The requirements presented in this section were derived from three sources: the research objectives stated in Section 1.5, the research gaps identified in Section 2.6, and the security layers proposed in Section 2.7. They are separated into functional requirements, which describe what the system must do, and non-functional requirements, which describe the quality attributes the system must exhibit. Each requirement is assigned a unique identifier so that it can be referenced by the design models presented in the remainder of this chapter and verified during evaluation.

3.3.1 Functional Requirements

The functional requirements define the observable capabilities of the RAGShield framework. They span the complete request lifecycle, from authentication and input inspection through retrieval and generation to output verification and auditing, and are summarised in Table 3.2.

Table 3.2: Functional Requirements of the RAGShield Framework

ID	Requirement	Description
FR-01	User authentication	The system shall authenticate every user through credential verification and, where enabled, multi-factor authentication before any session is established.
FR-02	Session management	The system shall issue a signed session token bound to the authenticated user and their assigned roles, and shall re-authenticate any request presenting an invalid or expired session.
FR-03	Brute-force protection	The system shall enforce rate limiting on authentication attempts and shall temporarily lock an account when the configured threshold is exceeded.
FR-04	User and role management	The System Administrator shall be able to create, modify, and remove user accounts and to assign the clinical and operational roles associated with each account.
FR-05	Security policy configuration	The System Administrator shall be able to define and modify role-based, attribute-based, and context-based access rules, subject to an automated consistency check for rule conflicts.
FR-06	Prompt submission	Authenticated clinical staff shall be able to submit natural-language prompts, upload medical documents, review generated responses, and view their prior interaction history.
FR-07	Prompt injection detection	The system shall analyse every submitted prompt for direct, indirect, and obfuscated injection attempts and shall assign a composite risk score in the range 0 to 100.
FR-08	Input sanitisation and re-verification	The system shall sanitise prompts of intermediate risk and re-verify the sanitised result, rejecting any prompt whose malicious payload survives sanitisation.
FR-09	Sensitive data detection and masking	The system shall detect PII, PHI, and PCI entities in prompts and retrieved content and shall replace them with reversible placeholder tokens that preserve contextual meaning.
FR-10	Query authorisation	The system shall evaluate every query against the applicable access policy before retrieval is performed, and shall deny queries that exceed the requester’s clearance, department, or contextual scope.
FR-11	Secure document ingestion	The system shall validate the type and size of every uploaded document, scan it for malware, extract its text, and screen the extracted content for poisoning and embedded instructions before acceptance.
FR-12	Document integrity sealing	The system shall compute and store a keyed hash (HMAC) for every indexed chunk so that any subsequent modification of knowledge-base content can be detected at retrieval time.
FR-13	Trust-aware retrieval	The retrieval firewall shall verify chunk integrity, evaluate document trust scores, and exclude untrusted, quarantined, or poisoned content from the context passed to the language model.
FR-14	Secure context construction and generation	The system shall assemble a sanitised prompt and verified context, invoke the external language model provider, and receive the draft response without exposing unmasked sensitive data.

ID	Requirement	Description
FR-15	Output verification	The system shall inspect every generated response for residual sensitive information, policy violations, and claims unsupported by the retrieved context, and shall block responses that fail verification.
FR-16	Controlled unmasking	The system shall restore masked values in the final response only to the extent permitted by the requester's authorisation, and shall attach source, confidence, and trust metadata to the delivered answer.
FR-17	Immutable audit logging	The system shall record every security-relevant event in an append-only audit log in which each entry stores its own hash and the hash of the preceding entry, forming a tamper-evident chain.
FR-18	Security monitoring and alerting	The system shall raise alerts for blocked prompts, quarantined documents, integrity failures, authorisation denials, and anomalous access patterns, and shall present them on an operational dashboard.
FR-19	Audit trail inspection and export	The Security Analyst shall be able to query the audit trail, verify its hash chain, correlate related events, and export a digitally signed investigation report.

3.3.2 Non-Functional Requirements

The non-functional requirements express the quality attributes that the framework must satisfy in order to be deployable in a real clinical environment. They are of particular importance to this project because Section 2.6.4 identified the trade-off between theoretically strong protection and practical deployability as one of the principal unresolved gaps in the literature. The non-functional requirements are summarised in Table 3.3.

Table 3.3: Non-Functional Requirements of the RAGShield Framework

ID	Quality Attribute	Design Requirement
NFR-01	Security	The architecture shall apply zero-trust principles throughout, such that no component, session, or document is trusted implicitly and every stage independently enforces its own controls.
NFR-02	Privacy and compliance	Sensitive health information shall remain protected in transit, at rest, and during processing, in a manner consistent with HIPAA and GDPR obligations for healthcare data.
NFR-03	Performance	The cumulative latency introduced by the security layers shall remain within limits acceptable for interactive clinical use, avoiding the prohibitive overhead associated with purely cryptographic retrieval.
NFR-04	Reliability	Security controls shall fail closed: a failure or timeout in any protective component shall result in rejection of the request rather than in unprotected processing.
NFR-05	Scalability	The framework shall accommodate growth in the number of concurrent users, indexed documents, and stored audit records without redesign of its core components.

ID	Quality Attribute	Design Requirement
NFR-06	Auditability	Every security decision shall be reconstructable after the fact from the audit trail, and any tampering with that trail shall be detectable through hash-chain verification.
NFR-07	Maintainability	The system shall follow Clean Architecture principles with strict separation of concerns, so that individual security layers can be modified or replaced independently.
NFR-08	Usability	Security enforcement shall remain transparent to clinical users during normal operation, and blocked or filtered responses shall be accompanied by an intelligible explanation.
NFR-09	Interoperability	The language model, embedding model, and vector store shall be accessed through defined interfaces so that alternative providers can be substituted without architectural change.
NFR-10	Portability	The framework shall be deployable within an institution's own infrastructure so that regulated data need not leave the organisational boundary except in sanitised form.

3.4 Functional Modelling (Use Case Diagram)

The use case model defines the functional boundary of RAGShield by identifying the actors that interact with the system and the services the system provides to each of them. It also makes explicit an important property of the proposed design: several security services are not optional features that an actor may choose to invoke, but mandatory steps that are automatically included in every relevant interaction.

3.4.1 System Actors

The framework recognises three categories of human actor and two external system actors. Human actors are modelled using generalisation, so that specialised roles inherit the use cases of their parent actor while remaining distinguishable for the purposes of authorisation. External actors represent services that lie outside the trust boundary of the system and whose responses are therefore treated as untrusted input. The actors and their responsibilities are summarised in Table 3.4.

Table 3.4: System Actors and Their Responsibilities

Actor	Type	Specialisation	Primary Responsibility
Staff	Human (primary)	Doctor, Nurse, Receptionist	Submits clinical prompts, uploads medical documents, and reviews generated responses within the limits of an assigned clearance level.

Actor	Type	Specialisation	Primary Responsibility
System Administrator	Human (primary)	—	Manages user accounts and roles and configures the access-control and security policies enforced by the framework.
Operator	Human (primary)	Security Analyst	Monitors system activity, investigates alerts, inspects the audit trail, and reviews detected injection and poisoning attempts.
LLM Provider	External system	—	Receives the sanitised prompt with its verified context and returns a draft response; treated as an untrusted component whose output is verified before delivery.
Authentication Service	External system	—	Performs identity verification and supports the multi-factor challenge issued during session establishment.

3.4.2 Use Case Analysis

Figure 3.1 presents the complete use case model. The diagram separates actor-initiated use cases, shown in the outer regions, from the internal security services drawn into every interaction through «include» relationships. This separation is the visual expression of the complete-mediation principle: an actor cannot reach the retrieval or generation stages without first traversing authentication, authorisation, input validation, and logging.

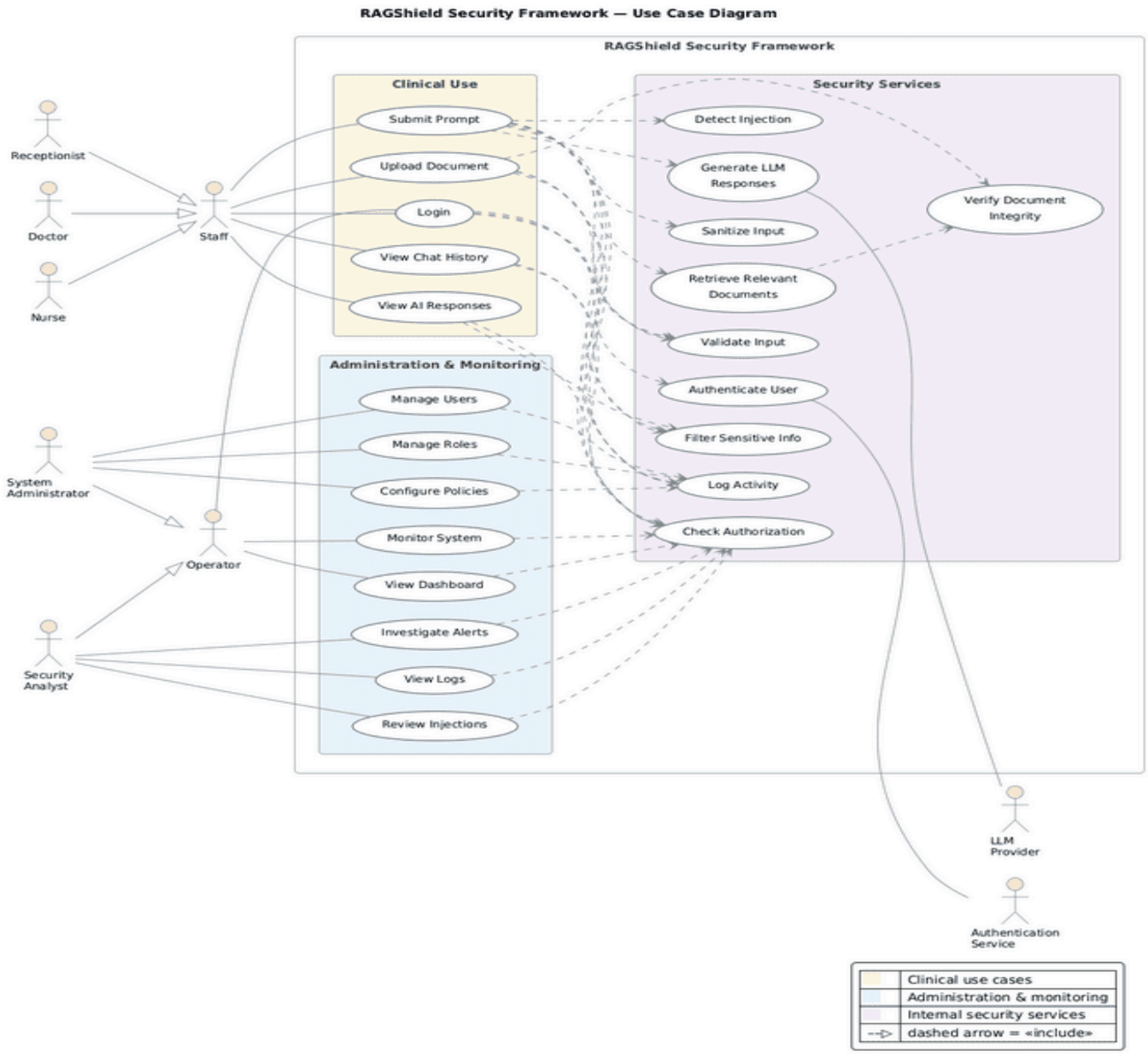

Figure 3.1: RAGShield Use Case Diagram

Clinical Staff Use Cases

Staff is a generalised actor representing the Doctor, Nurse, and Receptionist roles, each of which carries a distinct clinical clearance level. Following successful authentication, the activity of these roles centres on the clinical use module, in which a user may submit a prompt, upload a medical document, view previous chat history, and review AI-generated responses. Although the three specialisations share the same set of use cases, the information each may retrieve differs, because authorisation is evaluated against the clearance level and department attached to the individual role rather than against the generalised actor.

System Administrator Use Cases

The System Administrator is responsible for the administrative configuration of the framework. This comprises managing user accounts, managing roles and their associated permissions, and configuring the security policies that govern access to knowledge-base content. The administrator does not participate in clinical retrieval, which preserves a separation of duties between those who define access rules and those who exercise them.

Operator Use Cases

Operator is a generalised actor whose specialisation is the Security Analyst. The responsibilities of this role are oversight and investigation rather than configuration: monitoring the system in real time, viewing the operational dashboard, investigating security alerts, reviewing activity logs, and examining detected prompt injection attempts. Because the Security Analyst can read the audit trail, access to that trail is itself an audited event, as described in Section 3.6.2.

Internal Security Services

The internal security services are use cases that no actor invokes directly. They are triggered automatically through «include» relationships whenever a dependent use case is executed, and comprise authenticating the user, checking authorisation, sanitising and validating input, detecting injection attempts, filtering sensitive information, retrieving relevant documents, generating the language-model response, verifying document integrity, and logging activity. Modelling these services as mandatory inclusions rather than as optional extensions is what prevents any execution path from bypassing the security pipeline.

External Actors

The LLM Provider represents the external language model service invoked during response generation, while the Authentication Service represents the external system responsible for identity verification and multi-factor challenges. Both lie outside the system trust boundary. Consequently, data sent to the LLM Provider is sanitised beforehand and the response it returns is verified afterwards, in accordance with the zero-trust principle stated in Section 3.1.

3.5 System Workflow (Activity Diagrams)

The activity diagrams provide a control-flow view of the core business processes of the framework, showing the decision points at which a request may be accepted, corrected, or rejected. Three workflows are modelled, corresponding to the three operational domains of

RAGShield: administrative control, security monitoring, and clinical prompt processing. In all three, every terminating branch converges on a logging step, so that rejected and accepted requests alike leave an audit record.

3.5.1 Administrator Workflow

Figure 3.2 illustrates the workflow followed by the System Administrator when managing users and security policies. The process begins with the administrator logging into the dashboard and undergoing authentication through credentials and multi-factor verification. A failed attempt is logged and access is denied. Following successful authentication, the administrator enters the access control panel, at which point the workflow branches according to the selected action.

If **Manage Users** is selected, the system displays the current user list and permits the administrator to add, edit, or remove accounts and to assign roles such as Doctor, Nurse, or Receptionist. The entered data is then validated; an invalid entry produces a validation error and returns the administrator to the input step for correction. Once the data is valid, the changes are applied and persisted.

If **Manage Policies** is selected instead, the system displays the current RBAC, ABAC, and CBAC rules and allows the administrator to modify role-based, attribute-based, or context-based permissions together with the associated data access scope. The modified policy is submitted to a consistency check that detects potential rule conflicts before activation; a conflicting policy returns the administrator to the modification step. This check is significant from a security perspective because an inconsistent policy set is a common cause of unintended privilege escalation.

Both branches converge on a final logging step, in which all administrative actions are recorded by the audit and monitoring module before the process terminates.

Figure 3.2: Activity Diagram - System Administrator Workflow

3.5.2 Security Analyst Workflow

Figure 3.3 describes the workflow executed by the Security Analyst upon receiving a system alert, such as the detection of a poisoned document or an anomaly in a trust score. The analyst first reviews the alert details, including its source, severity, and the affected query, then consults the audit and monitoring logs and analyses the related query history in order to establish context. The contributing trust score factors are then examined: the source of the document, its digital signature, its owner information, the result of its integrity check, and its approval status.

The analyst then determines whether the alert represents a confirmed threat. If it is judged a false positive, the alert is marked as resolved and the findings are documented. If the threat is confirmed, the workflow branches on severity. A critical alert results in the document being rejected or quarantined, followed by a trust score update and escalation to the administrator. A moderate or low-severity alert is instead flagged for review, with the trust score updated accordingly. In both cases the findings are documented in an incident report and the final action is written to the audit and monitoring module before the process terminates.

Retaining the false-positive branch is deliberate. Documenting resolved alerts provides the feedback needed to tune detection thresholds over time, which addresses a practical limitation of purely automated poisoning detection noted in Section 2.4.2.

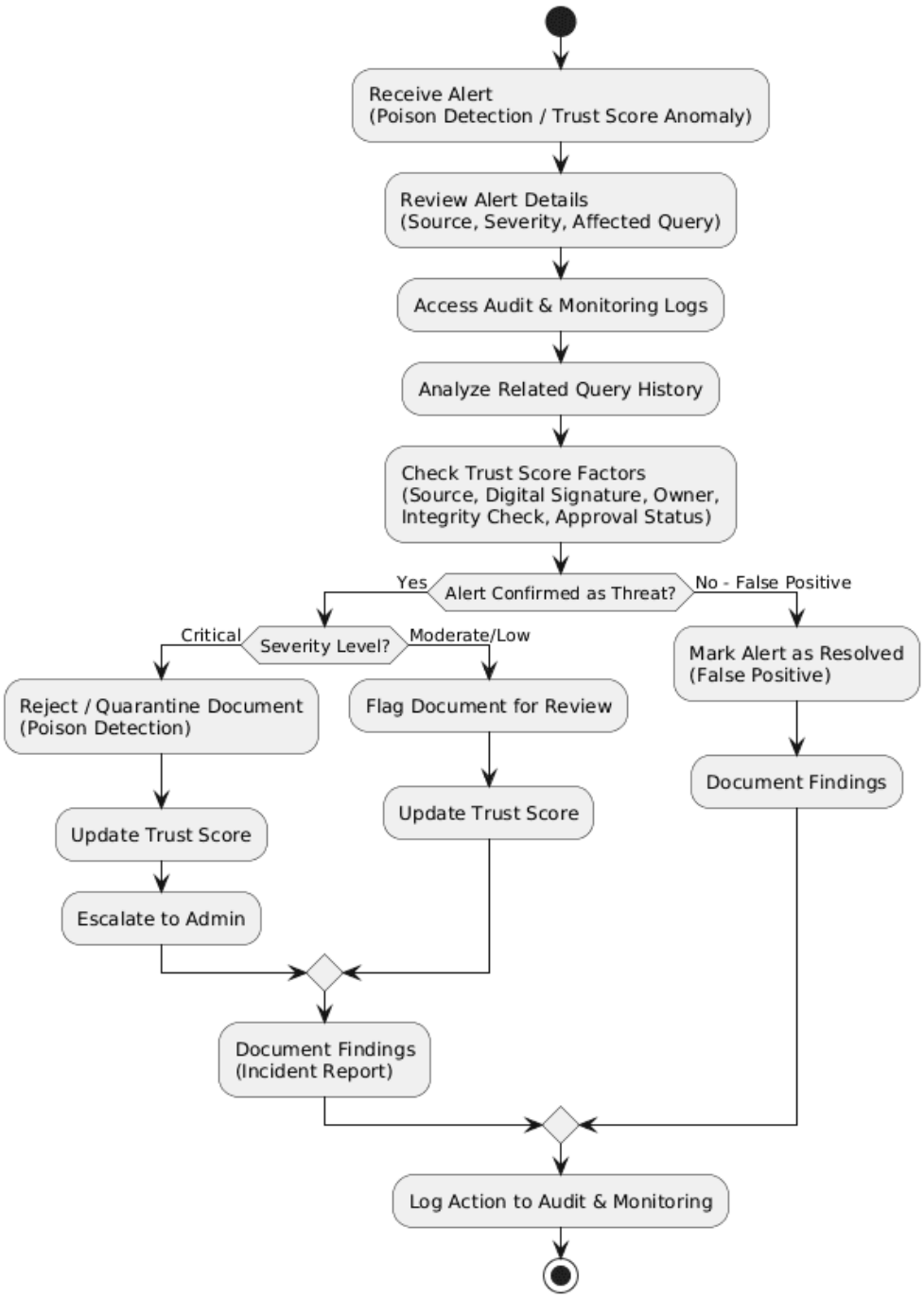

Figure 3.3: Activity Diagram - Security Analyst Workflow

3.5.3 Clinical Staff Workflow

Figure 3.4 represents the core end-to-end workflow of the RAGShield pipeline, tracing a clinical prompt from submission to final response. The process begins when the user enters a prompt, which passes through the input security firewall for risk assessment, producing a risk score between 0 and 100. A prompt judged unsafe is rejected and the rejection is logged;

otherwise the prompt proceeds to sensitive data detection and then to a policy and access control check based on the applicable RBAC and ABAC rules.

If the user is not authorised, access is denied and the event is logged. If the user is authorised, the system retrieves the relevant documents and applies context-preserving masking before passing them through the retrieval security engine, which performs poison detection and trust score evaluation. Untrusted documents cause the retrieval to be blocked and logged, whereas trusted documents are used to build the secure context.

The secure context is then used to generate a response through the language model. The generated response passes through the output verification engine, which performs hallucination detection and sensitive data leakage detection. A response found unsafe is blocked and logged. A response judged safe undergoes controlled unmasking, receives explainability metadata comprising its sources, confidence, and trust score, is recorded by the audit and monitoring module, and is finally delivered to the user, concluding the workflow.

This workflow demonstrates the layered structure of the framework in its clearest form. A request must satisfy five independent checks in sequence, and failure at any one of them terminates processing without reaching the next stage. An attack that defeats a single control therefore does not compromise the system, which is precisely the property that the fragmented approaches reviewed in Chapter Two were unable to provide.

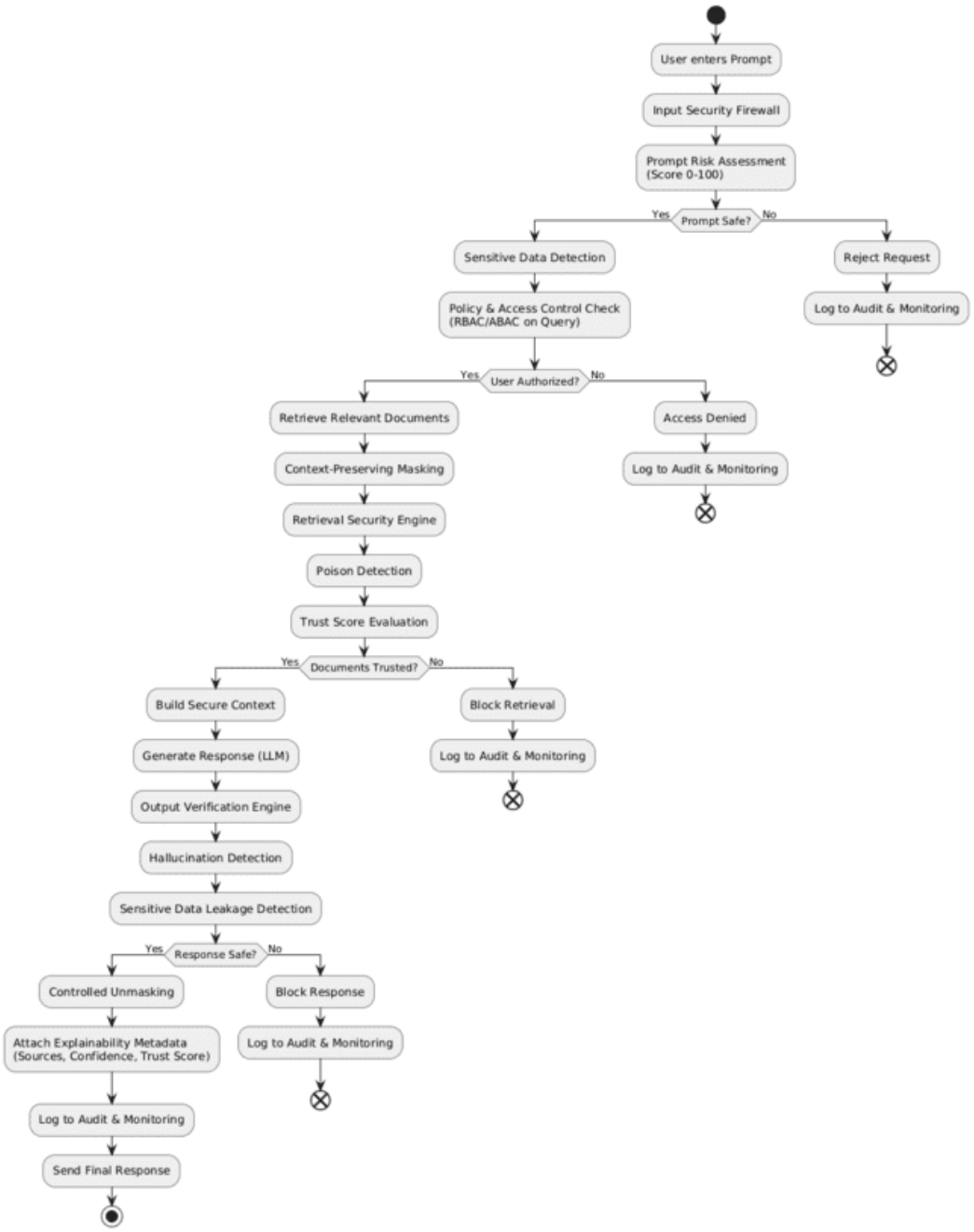

Figure 3.4: Activity Diagram - Clinical Staff Workflow

3.6 Dynamic Component Interaction (Sequence Diagrams)

The sequence diagrams present the detailed, layered message exchanges between system components for the most security-critical use cases, showing how each request is progressively validated, sanitised, authorised, and audited before reaching its outcome. Each diagram is organised into five numbered layers, and the same layer numbering is used

consistently across all four scenarios so that the corresponding stages can be compared directly.

3.6.1 Upload Knowledge Document

Figure 3.5 traces the anti-poisoning pipeline executed when a knowledge administrator uploads a document to the system. After the web application forwards the ingestion request, **Layer 1 (Session and Access Control)** verifies the session and checks authorisation for the WRITE_KB permission; unauthorised attempts are logged and rejected with an access-denied error. Once authorised, **Layer 2 (File and Content Screening)** validates the file type and size, scans for malware, extracts the text, and runs data-poisoning detection, rejecting unsupported or unsafe files and quarantining the document while raising a critical alert if injected or poisoned content is detected.

Accepted documents proceed to **Layer 3 (PHI/PII Protection)**, where any detected protected health information is de-identified. **Layer 4 (Chunking, Embedding and Integrity Sealing)** then splits the sanitised text into chunks, generates embeddings through the embedding service, and computes an HMAC for each chunk to provide tamper-evidence. Finally, **Layer 5 (Secure Persistence and Auditing)** stores the vectors and their HMACs in the vector database, verifies their integrity, logs the transaction, and confirms successful indexing to the administrator.

The significance of this sequence is that screening occurs before indexing rather than at retrieval time. Poisoned content is therefore prevented from entering the knowledge base at all, and the integrity seal applied in Layer 4 allows any later modification of stored content to be detected during retrieval.

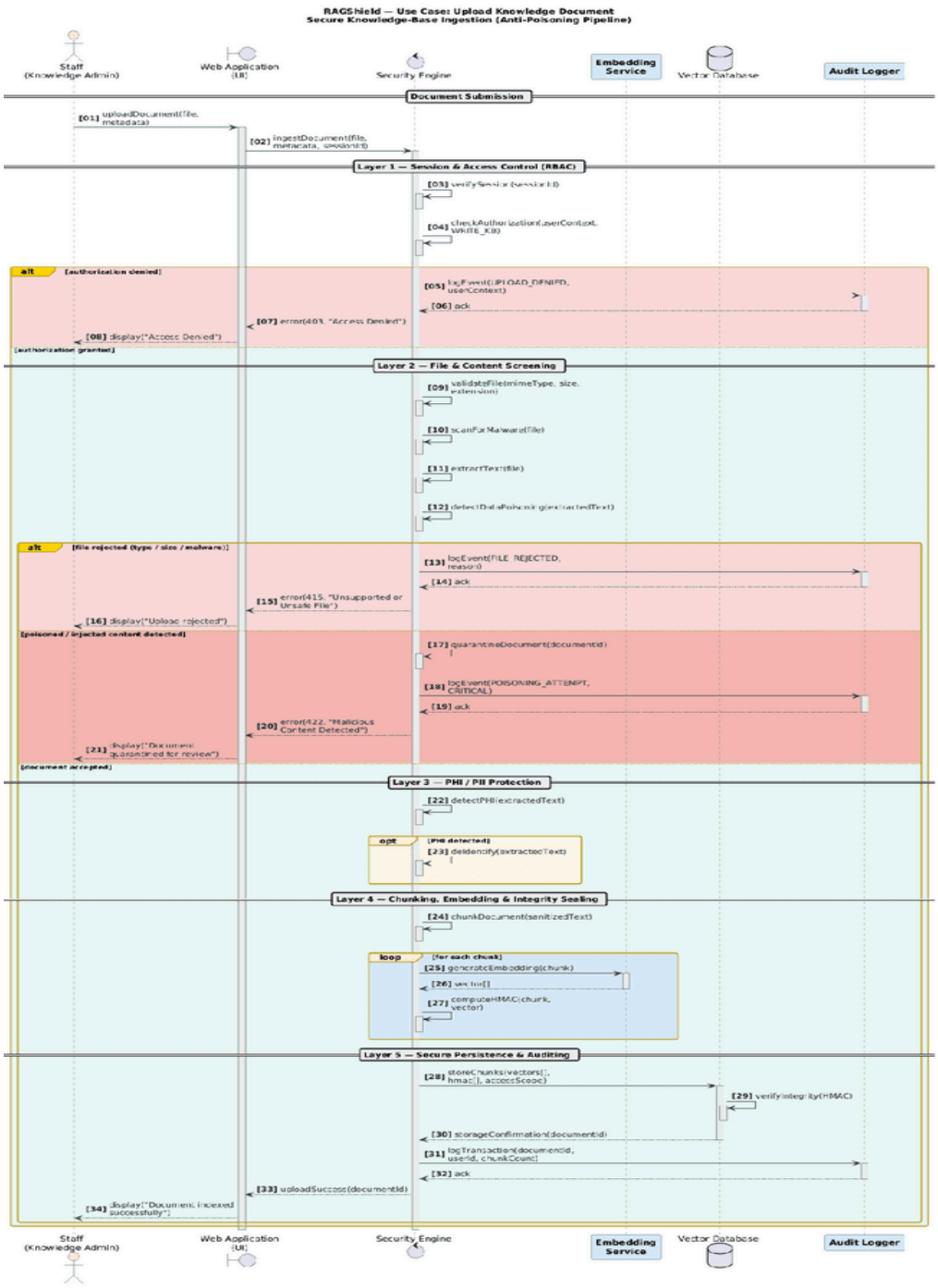

Figure 3.5: Sequence Diagram - Upload Knowledge Document

3.6.2 Review Security Logs

Figure 3.6 describes how a Security Analyst queries and inspects the audit trail of the system. Following session verification and authorisation for the READ_AUDIT permission in **Layer 1**, the filter criteria supplied by the analyst are sanitised in **Layer 2**, which prevents the investigation interface itself from becoming an injection vector. **Layer 3 (Log Retrieval and Integrity Verification)** then fetches the relevant entries from the append-only log store and

verifies their hash chain, raising a critical log tampering alert and displaying a warning if any integrity violation is detected.

Verified entries move to **Layer 4 (PHI Redaction and Correlation)**, where sensitive data is redacted, related events are correlated, and anomaly detection is applied, triggering a suspicious pattern alert when a threshold is exceeded. **Layer 5 (Meta-Audit and Presentation)** records the analyst's own access to the audit trail before displaying the report on the dashboard, and an optional export flow allows a digitally signed report to be generated for download.

Two aspects of this sequence deserve emphasis. First, the redaction step in Layer 4 ensures that the ability to investigate incidents does not become an unrestricted channel to patient data. Second, the self-referential audit step in Layer 5 extends the zero-trust principle to privileged users, so that the oversight function is itself subject to oversight.

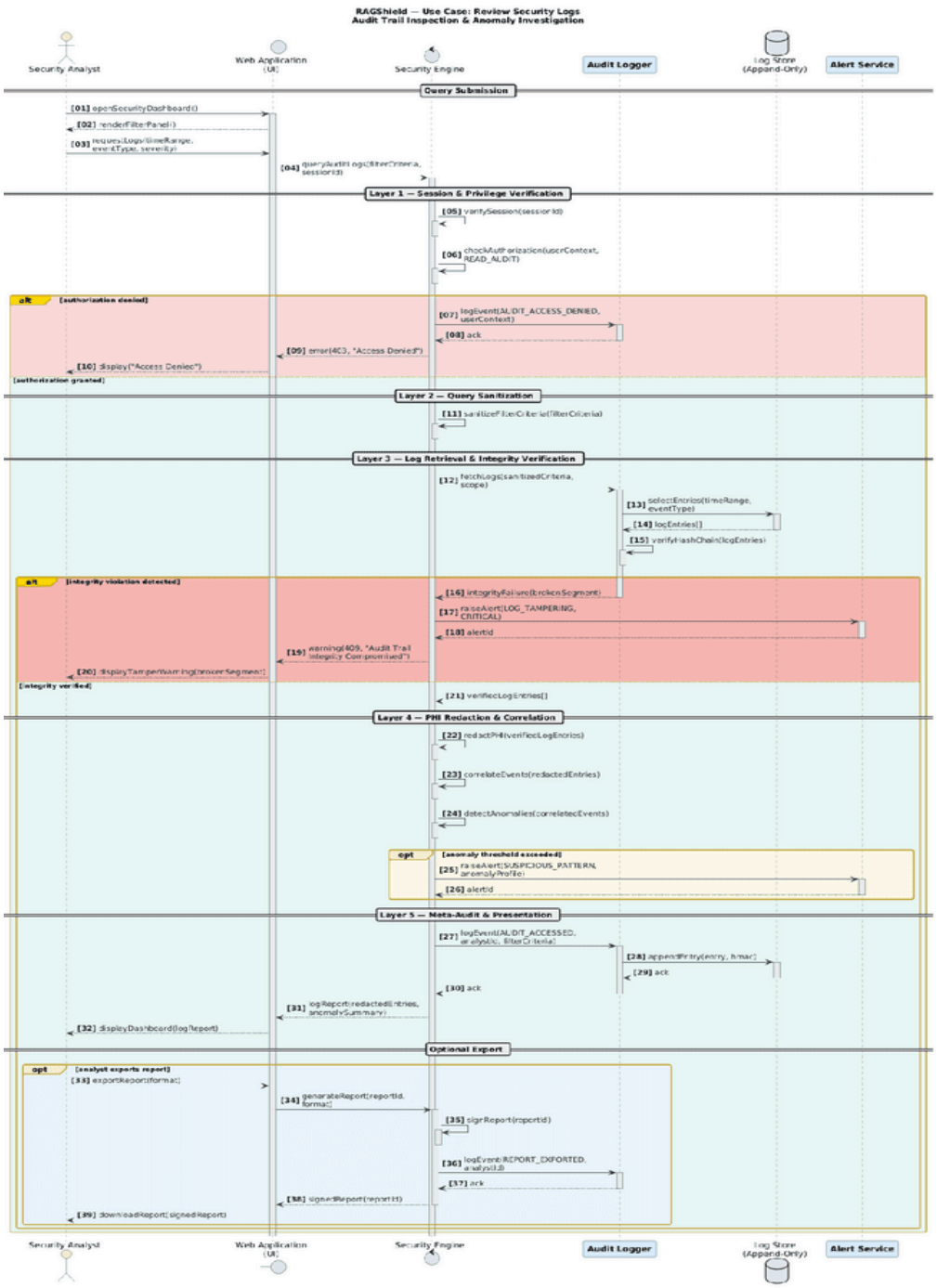

Figure 3.6: Sequence Diagram - Review Security Logs

3.6.3 Submit Prompt

Figure 3.7 presents the core sequence of the RAGShield platform, tracing a clinical prompt from submission to secured response. **Layer 1 (Input Validation and Session Verification)** validates the input and the session, invoking the authentication service to re-authenticate where the session is invalid; failed authentication terminates the sequence with no retrieval or language-model invocation performed. **Layer 2 (Prompt Injection Defence)** analyses the

prompt for injection attempts: high-confidence detections are blocked immediately, raise a critical alert, and increment the risk score associated with the user, while suspicious prompts of lower confidence undergo sanitisation and re-verification, following a fail-closed principle in which any payload surviving sanitisation is rejected rather than forwarded.

Clean prompts proceed to **Layer 3 (Access Control)**, which checks authorisation against the requested resource scope. **Layer 4 (Secure Context Retrieval)** then retrieves the relevant documents from the vector database and verifies their integrity through HMAC validation before the generation stage invokes the LLM provider to produce a draft response. Finally, **Layer 5 (Output Filtering and Auditing)** filters sensitive data from the response, records the complete transaction, and returns the secured response to the clinician.

The ordering of these layers is itself a design decision. Authorisation is evaluated before retrieval, so that unauthorised content is never loaded into memory, and integrity verification is performed after retrieval but before generation, so that tampered content cannot influence the model output.

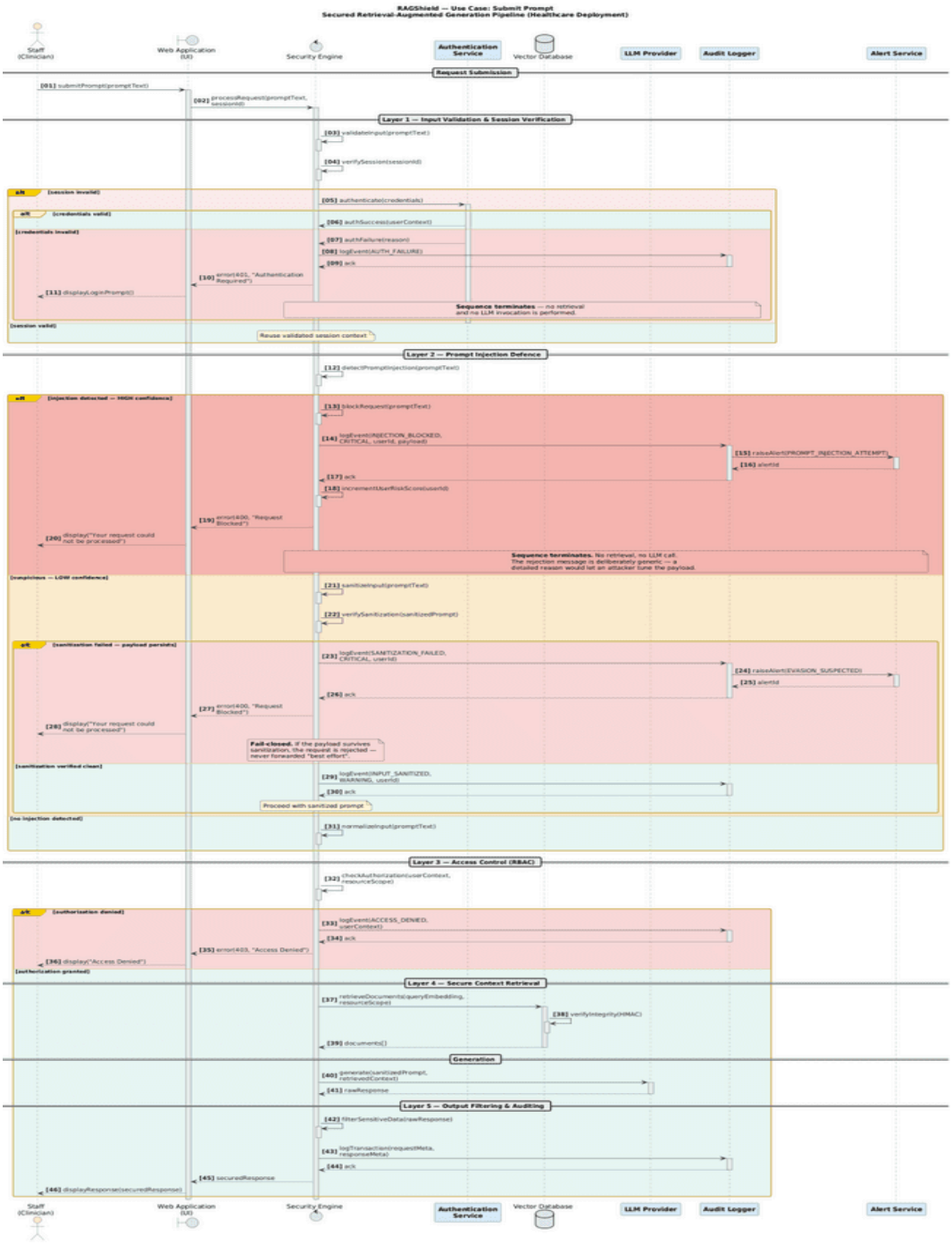

Figure 3.7: Sequence Diagram - Submit Prompt

3.6.4 Staff Login

Figure 3.8 details the multi-layered authentication flow for staff members. After credentials are submitted, **Pre-Authentication Controls** validate the input and enforce rate limiting, logging a brute-force suspicion event and temporarily locking the account if the configured limit is exceeded. **Credential Verification** then retrieves the user record from the identity store and compares the password hash, rejecting invalid credentials with a logged failure event.

Where valid credentials are presented, **Multi-Factor Authentication** is triggered if enabled: a one-time password challenge is issued and the user is permitted up to three verification attempts, each failure being logged. Exhausting all three attempts revokes the challenge, locks the account, and raises a possible MFA bypass alert, terminating the sequence without issuing a session token. Treating repeated one-time password failure as a potential attack signal rather than as routine user error reflects the zero-trust posture of the framework.

Once the one-time password is verified, **Session Establishment** issues a signed session token, creates a secure session bound to the roles of the user, records the successful login, and redirects the user to the dashboard. The role binding created at this point is the input on which every subsequent authorisation decision in Sections 3.5.1 to 3.5.3 depends.

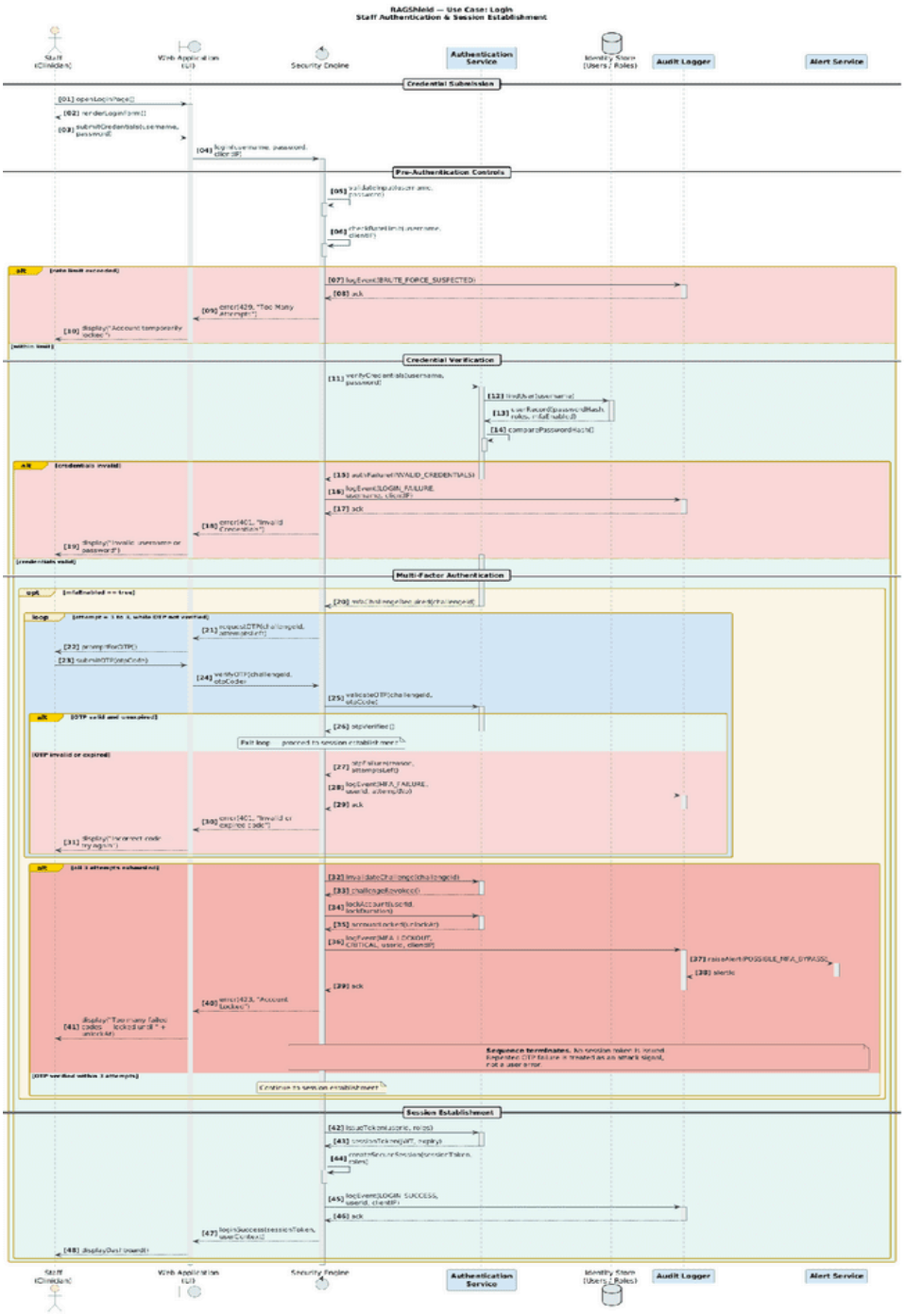

Figure 3.8: Sequence Diagram - Staff Login

3.7 Structural Design (Domain Class Diagram)

The class diagram shown in Figure 3.9 represents the object-oriented structure of the RAGShield domain layer and shows how the core entities of the system collaborate to enforce security and traceability.

The **User** entity is linked to **Role**, which determines whether access is clinical or operational and carries a clinical clearance level, while **AccessPolicy** governs the departments, IP ranges, countries, and time windows permitted for each role. Every incoming interaction is represented as a **RagRequest**, which is associated with a **MaskingSession** that manages the temporary **MaskedToken** value objects created to protect PII, PHI, and PCI data during processing. The **Document** and **DocumentChunk** classes track the sensitivity, digital signature, trust score, and poison status of the knowledge-base content used for retrieval.

Each request is evaluated by a **RiskAssessment**, which combines pattern-matching, machine-learning, and LLM-judge scores into a composite risk verdict, and every security-relevant action is permanently recorded as an **AuditLogEntry**, storing a cryptographic row hash together with a reference to the hash of the preceding entry so as to preserve a tamper-evident chain. Supporting enumerations such as RoleCategory, EntityType, ActionStatus, PoisonStatus, and RequestStatus standardise the permitted states across the system.

Modelling masked values as short-lived value objects owned by a masking session, rather than as attributes of the request, confines sensitive data to a bounded lifetime and a single point of control. This separation of concerns is consistent with Clean Architecture principles and supports the maintainability requirement NFR-07.

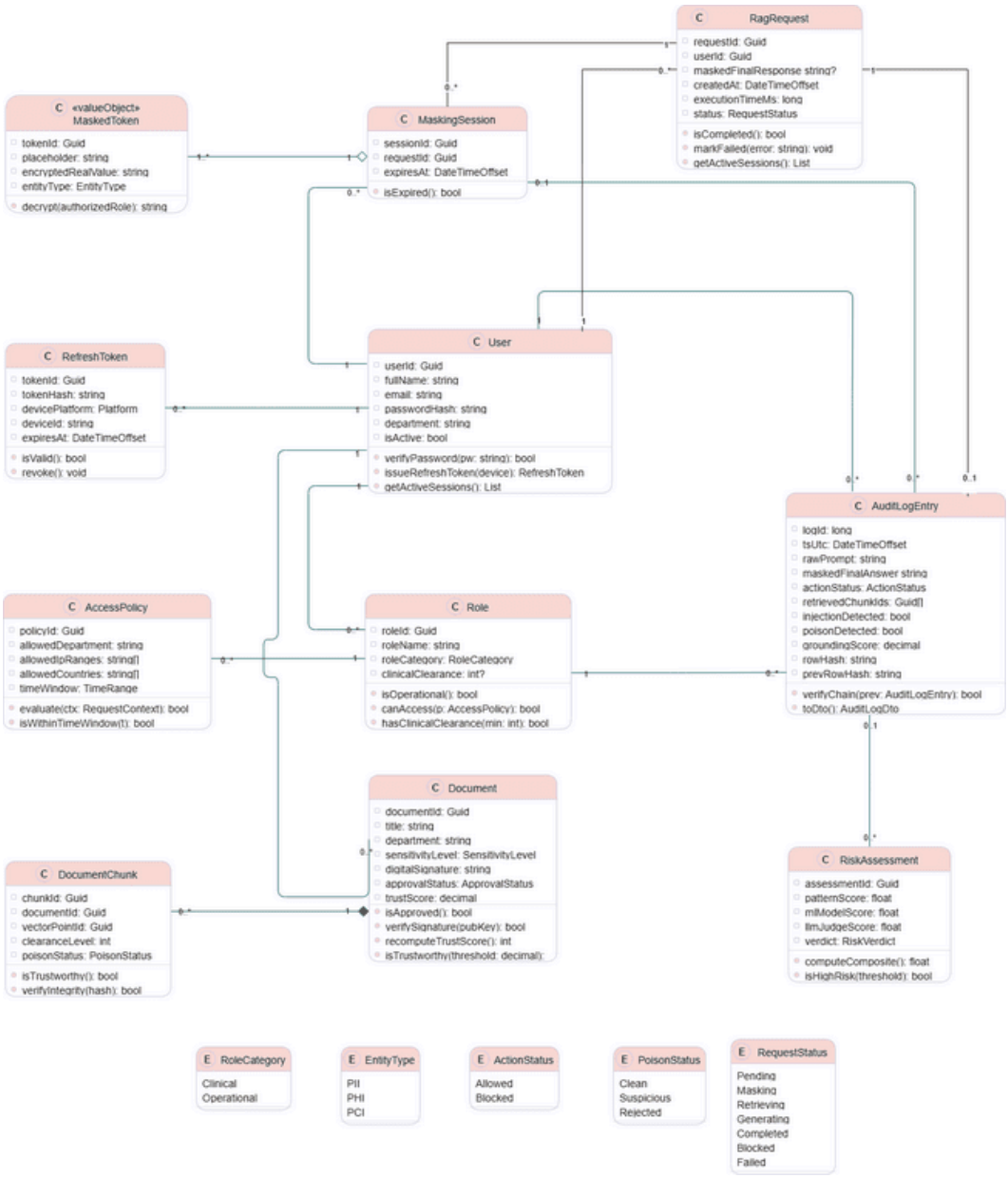

Figure 3.9: RAGShield Domain Class Diagram

3.8 Database Design (Entity-Relationship Diagram)

Figure 3.10 illustrates the relational database schema that supports the RAGShield system through SQL Server and Entity Framework Core. The schema realises the domain model of Section 3.7 in persistent form while adding the referential constraints required for forensic reconstruction of past events.

The **roles** table defines role categories, clinical clearance levels, and administrative permissions such as CanManagePolicies and CanViewAuditDashboard, and is linked to the **users** table, which stores the department and active status of each account. The

access_policies table extends roles with fine-grained RBAC and ABAC rules, including allowed departments, IP ranges, countries, and maximum clearance levels. Each user session that requires data protection creates a record in **masking_sessions**, linked in turn to the individual **masked_tokens** that hold the encrypted real value and entity type of every masked element.

The **documents** and **document_chunks** tables store the sensitivity level, approval status, trust score, and vector point identifiers of knowledge-base content, while **risk_assessments** records the composite risk score and verdict computed for each query. Every security-relevant event across these tables is consolidated into the **audit_log** table, which retains foreign keys to the user, role, session, and risk assessment involved, together with the PrevRowHash and RowHash fields that form an immutable, tamper-evident hash chain. The **refresh_tokens** table separately manages device-level authentication sessions for each user.

The design therefore maintains strict referential integrity while supporting both operational performance and forensic auditability, satisfying requirements FR-17 and NFR-06.

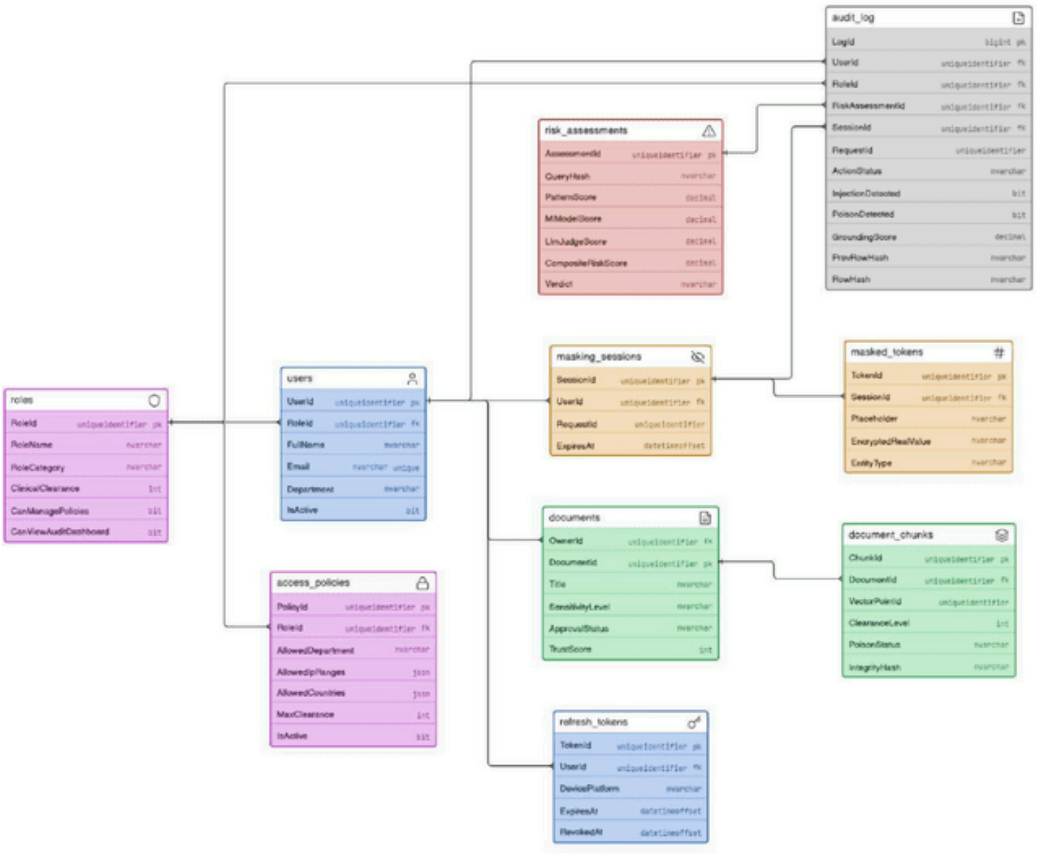

Figure 3.10: RAGShield Entity-Relationship Diagram

3.9 Data Flow Modelling

The data flow diagrams describe the movement of information into, through, and out of the framework. The context diagram establishes the external boundary of the system, and its decomposition exposes the internal processes that implement the defence-in-depth pipeline.

3.9.1 Context Diagram (Level 0)

Figure 3.11 provides a context-level overview of RAGShield, representing the framework as a single process that interacts with external entities across three boundaries. Within the **Users** boundary, Clinical Staff (E1) submit medical queries and patient prompts and receive secure unmasked responses, while Administrative Staff (E2) submit general operational queries and receive masked responses appropriate to their access level.

Within the **Administration** boundary, the Security Analyst (E3) sends threat queries and audit requests and receives security alerts, audit reports, and risk scores, while the System Administrator (E4) supplies access policies and system configuration and receives operational metrics and system status updates. On the **external systems** side, External Knowledge Sources (E5) provide raw healthcare documents and receive document processing status in return, and the External LLM (E6) receives the sanitised prompt with its secure context and returns the generated response.

The diagram makes visible a property that is central to the design: the only data leaving the system boundary towards the external language model is sanitised, and unmasked clinical information is returned exclusively to authorised clinical staff.

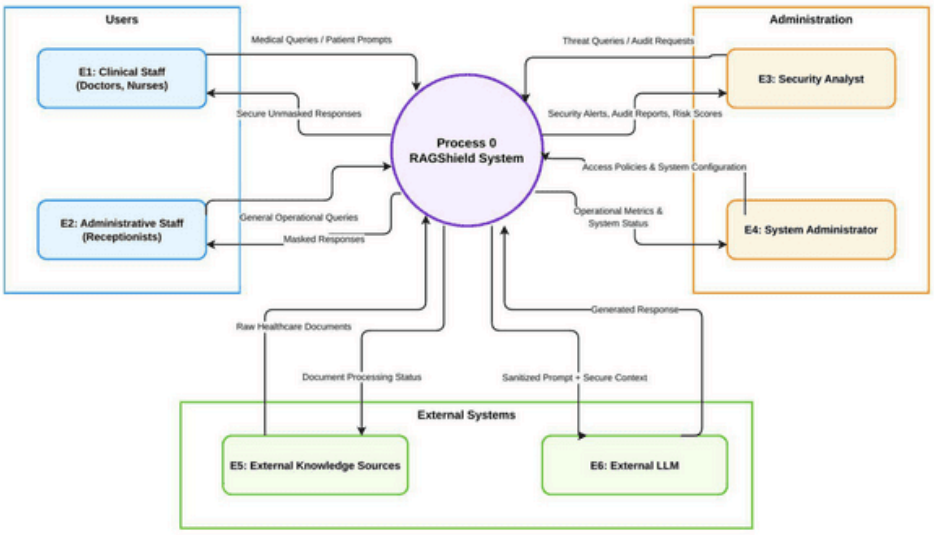

Figure 3.11: Data Flow Diagram (Level 0) - System Context

3.9.2 Detailed Data Flow (Level 1)

Figure 3.12 decomposes the context process into nine internal processes that together form the defence-in-depth pipeline of RAGShield. Incoming queries from Clinical Staff (E1) and Administrative Staff (E2) first reach **P1 (Input Security Firewall and Risk Assessment)**, which produces a risk-scored query and, where necessary, a blocked-prompt rejection. The query then moves to **P2 (Sensitive Data Detection and Masking)**, which sanitises the input and stores or retrieves the token mapping in the Masking Token Vault (D2), before passing the sanitised query to **P5 (Access Control and Query Authorisation)**, which evaluates it against the Access Policies and Trust Stores (D3) and issues an access-denied rejection where required.

Authorised queries proceed to **P6 (Retrieval Firewall and Trust Evaluation)**, which performs similarity search against the Encrypted Vector Database (D1) in order to build a trusted context. In parallel, **P3 (Document Ingestion, Chunking and Embedding)** and **P4 (Vector Database Management)** handle the intake and indexing of raw healthcare documents from External Knowledge Sources (E5) into that same vector store. The trusted context and

sanitised prompt are then sent to **P7 (Secure Context Builder and LLM Interface)**, which communicates with the External LLM (E6) and returns a draft response.

The draft passes through **P8 (Output Firewall and Hallucination Check)** to produce a filtered, safe response, which finally reaches **P9 (Controlled Unmasking, Audit and Response Delivery)**. P9 performs token lookups in order to unmask authorised content, delivers the secure response to the user, writes an immutable record into the Audit Log Store hash chain (D4), and supplies security alerts, audit reports, and operational data to the Security Analyst (E3) and System Administrator (E4), who in turn manage and update the access policies of the system.

The four data stores are deliberately separated. Keeping the masking token vault distinct from the vector database ensures that compromise of the indexed content does not by itself permit reidentification of the masked values it refers to.

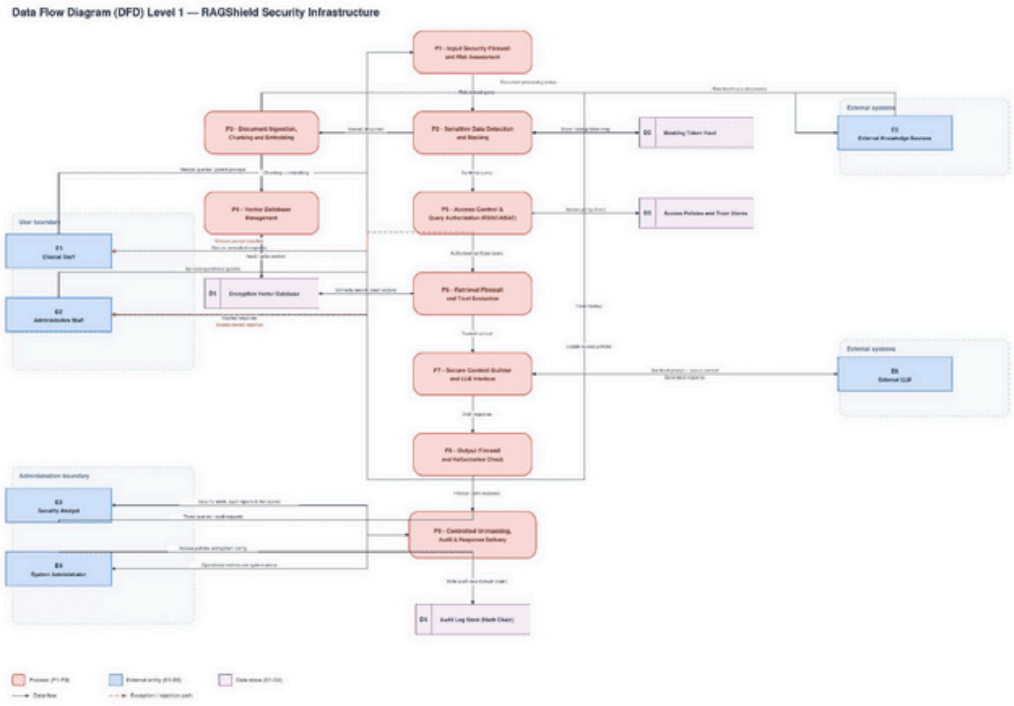

Figure 3.12: Data Flow Diagram (Level 1) - Security Processing Pipeline

3.10 Model Consistency Review

Because the design is expressed through several complementary models, the models were cross-checked against one another and against the written system description before the design was finalised. The check covered component naming, actor definitions, data flows, persistent entities, and the placement of security controls across the use case diagram, the

three activity diagrams, the four sequence diagrams, the domain class diagram, the entity-relationship diagram, and both levels of data flow diagram.

The models were found to be consistent in their treatment of the security layers: the seven layers proposed in Section 2.7 appear in every model in which they are applicable, in the same order, and under the same names. The component identifiers used in the Level 1 data flow diagram (P1 to P9, D1 to D4, and E1 to E6) are used consistently in the overall architecture described in Section 3.2 and in the narrative descriptions throughout the chapter.

Four points of divergence between models were identified during the check. They are recorded here rather than resolved unilaterally, because resolving them requires a decision about the intended design rather than a correction to a drawing.

3.10.1 Divergence Between the Actor Model and the Data Flow Model

[REVIEW WITH TA — diagram consistency issue] The use case diagram in Figure 3.1 and the actor definitions in Table 3.4 model a single primary human actor, Staff, with Doctor, Nurse, and Receptionist as specialisations of it. The data flow diagrams in Figures 3.11 and 3.12 instead model two separate external entities: E1, Clinical Staff (Doctors, Nurses), and E2, Administrative Staff (Receptionists). The receptionist role is therefore grouped with clinical users in the functional model but separated from them in the data flow model.

The divergence is consequential rather than cosmetic, because the two models imply different authorisation treatments. If receptionists are a specialisation of Staff, they share the clinical use cases and are distinguished only by clearance level; if they are a separate external entity, they follow a distinct data path and receive masked rather than unmasked responses, which is what Figure 3.11 shows. A decision is required as to which model is intended, after which the other should be brought into line with it.

3.10.2 Entities Present in the Class Diagram but Absent from the Schema

[REVIEW WITH TA — diagram consistency issue] The domain class diagram in Figure 3.9 defines a RagRequest class that carries the identifier, user, masked final response, timestamps, and status of each request. The entity-relationship diagram in Figure 3.10 contains no corresponding table, although a RequestId foreign key appears in both the masking_sessions and audit_log tables and therefore implies that such a table exists. Either the schema is missing the table that these foreign keys reference, or request state is intended to be held only in memory, in which case the foreign keys should not be modelled as references to a persistent entity.

3.10.3 Attribute Divergence Between the Class Diagram and the Schema

[REVIEW WITH TA — diagram consistency issue] The Document class in Figure 3.9 defines the attributes department and digitalSignature, together with a verifySignature operation. The documents table in Figure 3.10 contains neither attribute, holding only OwnerId, DocumentId, Title, SensitivityLevel, ApprovalStatus, and TrustScore. This matters for the integrity layer specifically: the cryptographic verification described in Section 2.7.2 and in Section 3.2.2 depends on a stored signature, and if the signature is not persisted then integrity cannot be verified after a restart. The department attribute is similarly required if attribute-based authorisation is to evaluate departmental policy against document ownership.

A comparable divergence exists in the audit trail. The AuditLogEntry class defines tsUtc, rawPrompt, and maskedFinalAnswer, none of which appears as a column in the audit_log table, even though the tamper-evident chaining described in Section 3.8 requires that the hashed content itself be recoverable for verification.

3.10.4 Placement of the Document Integrity Service in the Use Case Model

[REVIEW WITH TA — diagram consistency issue] In Figure 3.1 the Verify Document Integrity use case is drawn outside the Security Services grouping that contains the other internal security services, although Section 3.4.2 describes it as one of the internal services that no actor invokes directly. If it is an internal service triggered through «include» relationships, it should be positioned within the Security Services boundary alongside Detect Injection, Sanitize Input, Validate Input, Filter Sensitive Info, Check Authorization, and Log Activity.

3.10.5 Outcome of the Review

With the exception of the four points recorded above, the models were found to be mutually consistent and to agree with the written system description and with the overall architecture presented in Section 3.2. No diagram was redrawn as part of this review. Once the four points have been decided, the affected diagrams and the corresponding narrative should be updated together, and the traceability recorded in the following section rechecked.

3.11 Design Traceability

Section 2.7 proposed seven security layers in response to the research gaps identified in the literature. In order to demonstrate that the design presented in this chapter implements each of those layers rather than merely restating them, Table 3.5 maps every layer to the design models in which it is realised and to the requirements that it satisfies. This mapping also

provides the basis for the evaluation reported in the following chapters, since each layer can be tested against the requirements traced to it.

Table 3.5: Traceability Between Security Layers, Design Models and Requirements

Security Layer (Section 2.7)	Realised In	Requirements
Input security	Figures 3.1, 3.4, 3.7; process P1	FR-07, FR-08
Secure ingestion and integrity	Figures 3.5, 3.10; processes P3, P4	FR-11, FR-12
Privacy protection	Figures 3.4, 3.9, 3.12; process P2	FR-09, FR-16, NFR-02
Retrieval security and trust evaluation	Figures 3.3, 3.4, 3.7; process P6	FR-13
Dynamic access control	Figures 3.1, 3.2, 3.10; process P5	FR-04, FR-05, FR-10
Output validation	Figures 3.4, 3.7; process P8	FR-15
Continuous monitoring and auditing	Figures 3.3, 3.6, 3.10; process P9	FR-17, FR-18, FR-19, NFR-06

3.12 Chapter Summary

This chapter presented the analysis and design of the proposed RAGShield framework, translating the research gaps established in Chapter Two into an implementable technical specification. The chapter began by defining nineteen functional requirements covering the complete request lifecycle and ten non-functional requirements expressing the quality attributes necessary for deployment in a clinical environment.

The functional model was then presented through a use case diagram that identified three categories of human actor, two external system actors, and a set of internal security services drawn into every interaction through mandatory inclusion relationships. Three activity diagrams described the administrative, monitoring, and clinical workflows, and four sequence diagrams exposed the layered message exchanges underlying document ingestion, audit trail inspection, prompt processing, and authentication.

The structural design was expressed through a domain class diagram and a corresponding entity-relationship schema, both of which embed security attributes such as trust score, poison status, sensitivity level, and tamper-evident hash chaining directly into the data model rather than treating them as external annotations. The data flow models then described the movement of information across the system boundary and the nine internal processes that implement the defence-in-depth pipeline.

The models were then cross-checked against one another. The review confirmed that the seven security layers proposed in Section 2.7 appear consistently across the models and that component naming agrees with the overall architecture, while recording four points of divergence that require a design decision before submission. Taken together, these models show that the seven security layers are realised as an integrated architecture rather than as a collection of independent controls, and Table 3.5 established explicit traceability between each layer, the models that realise it, and the requirements it satisfies. The following chapter describes the implementation of this design, including the technologies selected for each layer and the mechanisms used to enforce the security properties specified here.

References

References are listed in IEEE style in the order of their first citation in the text. This list currently contains only those sources that are cited in the report and whose bibliographic details have been verified. It is not yet complete: see the note at the end of this section.

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] D. Karem and S. Osman, “ArabGuard: A prompt injection detection framework for Arabic, Egyptian dialect and Franco-Arabic input,” 2026. [Online]. Available: https://huggingface.co/d12o6aa/ArabGuard

[3] W. Zou, R. Geng, B. Wang, and J. Jia, “PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models,” in *Proc. 34th USENIX Security Symposium (USENIX Security 25)*, Seattle, WA, USA, Aug. 2025, pp. 3827–3844.

[4] H. Zhou, K.-H. Lee, Z. Zhan, Y. Chen, Z. Li, Z. Wang, H. Haddadi, and E. Yilmaz, “TrustRAG: Enhancing robustness and trustworthiness in retrieval-augmented generation,” arXiv preprint arXiv:2501.00879, 2025.

[5] A. Bassit and V. Boddeti, “SecureRAG: End-to-end secure retrieval-augmented generation,” in *Proc. 2nd Workshop on GenAI for Health: Potential, Trust, and Policy Compliance*, 2025.

[6] S. Zeng, J. Zhang, P. He, Y. Xing, Y. Liu, H. Xu, J. Ren, S. Wang, D. Yin, Y. Chang, and J. Tang, “The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG),” in *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, Aug. 2024, pp. 4505–4524.

[7] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” arXiv preprint arXiv:2401.15884, 2024.

[8] O. Adjali, O. Ferret, S. Ghannay, and H. Le Borgne, “Multi-level information retrieval augmented generation for knowledge-based visual question answering,” in *Proc. 2024*

Conference on Empirical Methods in Natural Language Processing (EMNLP), Miami, FL, USA, Nov. 2024, pp. 16499–16513.

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[10] S. Aboukadri, A. Ouaddah, A. Mezrioui, and I. El Asri, “Leveraging RAG and LLMs for access control policy extraction from user stories in agile software development,” *IEEE Access*, vol. 13, pp. 116462–116472, 2025.

[11] X. Xu, H. Weytjens, D. Zhang, Q. Lu, I. Weber, and L. Zhu, “RRAGOps: Operating and managing retrieval-augmented generation pipelines,” 2025.

[12] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal, “Certifiably robust RAG against retrieval corruption,” arXiv preprint arXiv:2405.15556, 2024.

[13] P. Cheng, Y. Ding, T. Ju, Z. Wu, W. Du, P. Yi, Z. Zhang, and G. Liu, “TrojanRAG: Retrieval-augmented generation can be backdoor driver in large language models,” arXiv preprint arXiv:2405.13401, 2024.

[14] J. Xue, M. Zheng, Y. Hu, F. Liu, X. Chen, and Q. Lou, “BadRAG: Identifying vulnerabilities in retrieval augmented generation of large language models,” arXiv preprint arXiv:2406.00083, 2024.

[15] Y. Zhang, Q. Li, T. Du, X. Zhang, X. Zhao, Z. Feng, and J. Yin, “HijackRAG: Hijacking attacks against retrieval-augmented large language models,” arXiv preprint arXiv:2410.22832, 2024.

[16] A. Shafran, R. Schuster, and V. Shmatikov, “Machine against the RAG: Jamming retrieval-augmented generation with blocker documents,” 2025.

[17] C. Yan, B. Guan, Y. Li, M. H. Meng, L. Wan, and G. Bai, “Understanding and detecting file knowledge leakage in GPT app ecosystem,” in *Proc. ACM*, 2025. doi: 10.1145/3696410.3714755.

[18] L. Ammann, S. Ott, C. R. Landolt, and M. P. Lehmann, “Securing RAG: A risk assessment and mitigation framework,” in *Proc. 2025 IEEE Swiss Conference on Data Science (SDS)*, 2025, pp. 127–134.

[19] Y. Liu, X. Zhao, D. Song, and Y. Bu, “Dataset protection via watermarked canaries in retrieval-augmented LLMs,” arXiv preprint arXiv:2502.10673, 2025.

[20] B. Kitchenham and S. Charters, “Guidelines for performing systematic literature reviews in software engineering,” EBSE Technical Report EBSE-2007-01, Keele University and University of Durham, 2007.

[21] M. J. Page *et al*., “The PRISMA 2020 statement: An updated guideline for reporting systematic reviews,” *BMJ*, vol. 372, p. n71, 2021.

[22] E. Hilario, S. Azam, J. Sundaram, K. I. Mohammed, and B. Shanmugam, “Generative AI for pentesting: The good, the bad, the ugly,” *International Journal of Information Security*, vol. 23, pp. 2075–2097, 2024, doi: 10.1007/s10207-024-00835-x.

[23] Jayanth, “Mission-Pumpkin v1.0: PumpkinFestival,” VulnHub VM Challenge, 2019. [Online]. Available: https://www.vulnhub.com/entry/mission-pumpkin-v10-pumpkinfestival,329/

[24] Q. Liu, W. Mo, T. Tong, J. Xu, F. Wang, C. Xiao, and M. Chen, “Mitigating backdoor threats to large language models: Advancement and challenges,” in *Proc. 60th Annual Allerton Conference on Communication, Control, and Computing*, 2024, pp. 1–8.

[25] Y. LeCun, C. Cortes, and C. Burges, “MNIST handwritten digit database,” 1998. [Online]. Available: http://yann.lecun.com/exdb/mnist/

[26] GTSRB, “German Traffic Sign Recognition Benchmark dataset,” 2011. [Online]. Available: https://benchmark.ini.rub.de/gtsrb_dataset.html

[27] P. Zhou, Y. Feng, and Z. Yang, “Privacy and security challenges in large language models,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 2, pp. 297–307, 2024.

[28] W. Lu, Z. Zeng, J. Wang, Z. Lu, Z. Chen, H. Zhuang, and C. Chen, “AdvBench: A benchmark for jailbreaking attacks on large language models,” Hugging Face Dataset, 2024. [Online]. Available: https://huggingface.co/datasets/walledai/AdvBench

[29] G. Research, “Natural Questions dataset,” 2019. [Online]. Available: https://ai.google.com/research/NaturalQuestions

[30] Z. Yang, Z. Qi, and W. T. Yih *et al*., “HotpotQA: A dataset for diverse, explainable multi-hop question answering,” 2018. [Online]. Available: https://hotpotqa.github.io/

[31] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary *et al*., “MS MARCO: A human-generated machine reading comprehension dataset,” 2016. [Online]. Available: https://github.com/microsoft/msmarco

[32] phycholosogy, “ChatDoctor dataset,” 2025. [Online]. Available: https://github.com/phycholosogy/RAG-privacy/tree/main/Data

[33] phycholosogy, “Enron mail dataset,” 2025. [Online]. Available: https://github.com/phycholosogy/RAG-privacy/tree/main/Data

[34] Z. Liu, P. Luo, X. Wang, and X. Tang, “CelebA: Large-scale CelebFaces attributes dataset,” 2015. [Online]. Available: http://mmlab.ie.cuhk.edu.hk/projects/CelebA.html

[35] S. Schulhoff, J. Pinto, A. Khan, L.-F. Bouchard, C. Si *et al*., “HackAPrompt dataset,” 2023. [Online]. Available: https://huggingface.co/datasets/hackaprompt/hackaprompt-dataset

[36] C. Clop and Y. Teglia, “Backdoored retrievers for prompt injection attacks on retrieval augmented generation of large language models,” arXiv preprint arXiv:2410.14479, 2024.

[37] A. Parrish, A. Chen, N. Nangia, V. Padmakumar, J. Phang *et al*., “BBQ: A hand-built bias benchmark for question answering,” 2022. [Online]. Available: https://github.com/nyu-mll/BBQ

[38] C. Lee, J. Kim, J. S. Lim, and D. Shin, “Generative AI risks and resilience: How users adapt to hallucination and privacy challenges,” *Telematics and Informatics Reports*, vol. 19, p. 100221, 2025.

[39] F. Dalpiaz, “Requirements data sets (user stories),” 2018. [Online]. Available: https://data.mendeley.com/datasets/7zbk8zsd8y/2

[40] B. Zhang, H. Xin, M. Fang, Z. Liu, B. Yi, T. Li, and Z. Liu, “Traceback of poisoning attacks to retrieval-augmented generation,” in *Proc. ACM SIGSAC Conference on Computer and Communications Security*, 2025.

[41] B. Mathew, P. Saha, S. M. Yimam, C. Biemann, P. Goyal, and A. Mukherjee, “HateXplain: A benchmark dataset for explainable hate speech detection,” 2021. [Online]. Available: https://github.com/hate-alert/HateXplain

[42] Y. Nie, A. Williams, E. Dinan, M. Bansal, J. Weston, and D. Kiela, “Adversarial NLI: A new benchmark for natural language understanding,” 2019. [Online]. Available: https://github.com/facebookresearch/anli

[43] H. Zhong, M. Lentz, N. Narodytska, A. Szekeres, and K. Rong, “HoneyBee: Efficient role-based access control for vector databases via dynamic partitioning,” 2025.

[44] Cohere, “Wikipedia 22-12 dataset,” 2022. [Online]. Available: https://huggingface.co/datasets/Cohere/wikipedia-22-12

[45] G. Byun, S. Lee, N. Choi, and J. D. Choi, “Secure Multifaceted-RAG for enterprise: Hybrid knowledge retrieval with security filtering,” *Information*, vol. 16, no. 9, p. 804, 2025.

[46] M. Xi, S. Lv, Y. Jin, G. Cheng, N. Wang, Y. Li, and J. Yin, “RIPRAG: Hack a black-box retrieval-augmented generation question-answering system with reinforcement learning,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.

[47] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits, “MedQA,” 2021. [Online]. Available: https://huggingface.co/datasets/bigbio/med_qa

[48] P. Zhou, Y. Feng, and Z. Yang, “Provably secure retrieval-augmented generation,” arXiv preprint arXiv:2508.01084, 2025.

[49] R. Wang, “HealthcareMagic-100k-en,” 2023. [Online]. Available: https://huggingface.co/datasets/wangrongsheng/HealthCareMagic-100k-en

[50] A. Kornilova and V. Eidelman, “BillSum: A corpus for automatic summarization of U.S. Congressional bills,” 2019. [Online]. Available: https://huggingface.co/datasets/billsum

[51] B. Zhang, Y. Chen, M. Fang, Z. Liu, L. Nie, T. Li, and Z. Liu, “Practical poisoning attacks against retrieval-augmented generation,” arXiv preprint arXiv:2504.03957, 2025.

[52] C. Jiang, X. Pan, G. Hong, C. Bao, Y. Chen, and M. Yang, “Feedback-guided extraction of knowledge base from retrieval-augmented LLM applications,” arXiv preprint arXiv:2411.14110, 2025.

[53] A. Arzanipour, R. Behnia, R. Ebrahimi, and K. Dutta, “RAG security and privacy: Formalizing the threat model and attack surface,” arXiv preprint arXiv:2509.20324, 2025.

[54] P. Cheng, Z. Zhang, and S. Wang, “Design and implementation of a secure RAG-enhanced AI chatbot for smart tourism customer service: Defending against prompt injection attacks,” arXiv preprint arXiv:2509.21367, 2025.

[55] S. Chaturvedi, G. Bagwe, L. Zhang, and X. Yuan, “AIP: Subverting retrieval-augmented generation via adversarial instructional prompt,” arXiv preprint arXiv:2509.15159, 2025.

[56] G. D. Stefano, L. Schönherr, and G. Pellegrino, “RAG and Roll: An end-to-end evaluation of indirect prompt manipulations in LLM-based application frameworks,” arXiv preprint arXiv:2408.05025, 2024.

[57] J. McHugh, K. Sekrst, and J. Cefalu, “Prompt injection 2.0: Hybrid AI threats,” arXiv preprint arXiv:2507.13169, 2025.

[58] X. Hu, X. Li, J. Chen, Y. Li, Y. Li, X. Li, Y. Wang, Q. Liu, L. Wen, P. S. Yu, and Z. Guo, “Evaluating robustness of generative search engines on adversarial factual questions,” arXiv preprint arXiv:2403.12077, 2024.

[59] I. Granica Computing, “Granica Screen: Data privacy service for generative AI and RAG pipelines,” 2024. [Online]. Available: https://docs.granica.ai/granica-screen-overview/

[60] Guardrails AI, “Guardrails: Input/output guards for reliable AI applications,” 2025. [Online]. Available: https://www.guardrails.io/

[61] Protect AI, “LLM Guard: Secure your LLM applications,” 2025. [Online]. Available: https://protectai.com/llm-guard

[62] L. Derczynski, E. Galinkin, J. Martin, S. Majumdar, and N. Inie, “garak: Generative AI red-teaming assessment kit,” 2024. [Online]. Available: https://garak.ai/

[63] L. Contributors, “Langfuse: Open-source LLM engineering platform for observability, prompt management and metrics,” 2025. [Online]. Available: https://langfuse.com/docs

[64] D. Rall, B. Bauer, M. Mittal, and T. Fraunholz, “Exploiting web search tools of AI agents for data exfiltration,” arXiv preprint arXiv:2510.09093, 2025.

[65] A. Castagnaro, U. Salviati, M. Conti, L. Pajola, and S. Pizzi, “The hidden threat in plain text: Attacking RAG data loaders,” 2025.